M.TECH · INTERNET PROGRAMMING USING JAVA

Java — Visual Study Book

Core Java → Servlets & JSP → MVC, RMI & EJB.
Every topic explained with a diagram, a real-life use case, code and an exam tip.

Unit I · Core Java | Unit II · OOP, Threads, JDBC | Unit III · HTTP & Servlets | Unit IV · JSP & DB apps | Unit V · MVC, RMI, EJB

33 original diagrams · definitions to open every answer · Hinglish memory hooks · revision checklist

Table of Contents

How to use this book 3

Review of Java Concepts 5

1.1 Features of Java 5
1.2 Object-Oriented Programming Overview 7
1.3 Introduction to Java Technologies — JDK, JRE, JVM 8
1.4 Writing a Simple Java Program 8
1.5 Data Types 9
1.6 Variables and Memory Concepts 10
1.7 Control Statements 10
1.8 Looping Constructs 11
1.9 Method Call Stack and Activation Records 11
1.10 Argument Promotion and Casting 12
1.11 Scope of Declaration and Method Overloading 13
1.12 String Handling 14
1.13 Arrays 15

Inheritance, Packages, Exceptions, Threads, Applets & JDBC 17

2.1 Inheritance: Extending Classes 17
2.2 Packages and Interfaces 18
2.3 Exception Handling 20
2.4 Multithreading 22
2.5 Applets 24
2.6 Database Connectivity: JDBC 25

HTTP, Web Servers & Servlets 28

3.1 Introduction to HTTP, Web Servers and Application Servers 28
3.2 Java Servlets 30
3.3 Servlet Packages, Classes, Interfaces and Methods 32
3.4 Handling Forms with a Servlet 33
3.5 Session Handling — Various Methods 34
3.6 Elements of Deployment Descriptors 35

JSP & Database-Backed Web Applications 37

4.1 JSP Basics 37
4.2 JSP / Servlet Database Connectivity — Oracle, MS-SQL, MySQL 40
4.3 Separating Business Logic and Presentation Logic 41
4.4 Session Handling in JSP 42
4.5 Types of Errors and Exception Handling (JSP) 43

MVC, RMI & EJB 44

5.1 MVC Architecture 44
5.2 Introduction to Remote Method Invocation (RMI) 45
5.3 Introduction to Enterprise JavaBeans (EJB) 47

Quick comparison sheet 50

Exam-day revision checklist 51

Recommended resources 51

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give you this ready-made).
2. **Explain in 3–5 lines** — plain English, use the keywords in **bold**.
3. **Diagram or code** — draw the figure from the book; examiners give marks for neat labelled diagrams.

Box guide

Box	What it gives you
📘 Definition	Opening line for the answer
🏭 Real-life use case	Where this is used in real projects — good for "applications" and viva questions
💡 Why it matters	The reason a feature exists (examiners love "why")
⭐ Exam tip	Marks-scoring points, what to memorise exactly
🧠 Yaad rakho	Short Hinglish memory hook
⚠️ Careful	Common mistakes and traps
❓ Likely exam questions	Past-paper style questions for that unit

Syllabus map

Unit	Hours	Focus	Main figures to practise
I	8	Core Java refresher — syntax, control flow, strings, arrays	JDK/JRE/JVM · JVM memory · call stack · String pool
II	8	Inheritance, packages, exceptions, multithreading, applets, JDBC	Exception hierarchy · thread life cycle · JDBC architecture
III	8	HTTP, web & application servers, servlets	Servlet life cycle · session tracking · web.xml mapping
IV	8	JSP, database connectivity, JavaBeans	JSP life cycle · implicit objects · layer separation
V	8	MVC, RMI, EJB	MVC flow · RMI architecture · EJB types

The big picture — how all five units fit together

A typical enterprise Java application uses **every** unit of this syllabus. Keep this picture in mind while reading; each later unit zooms into one box.

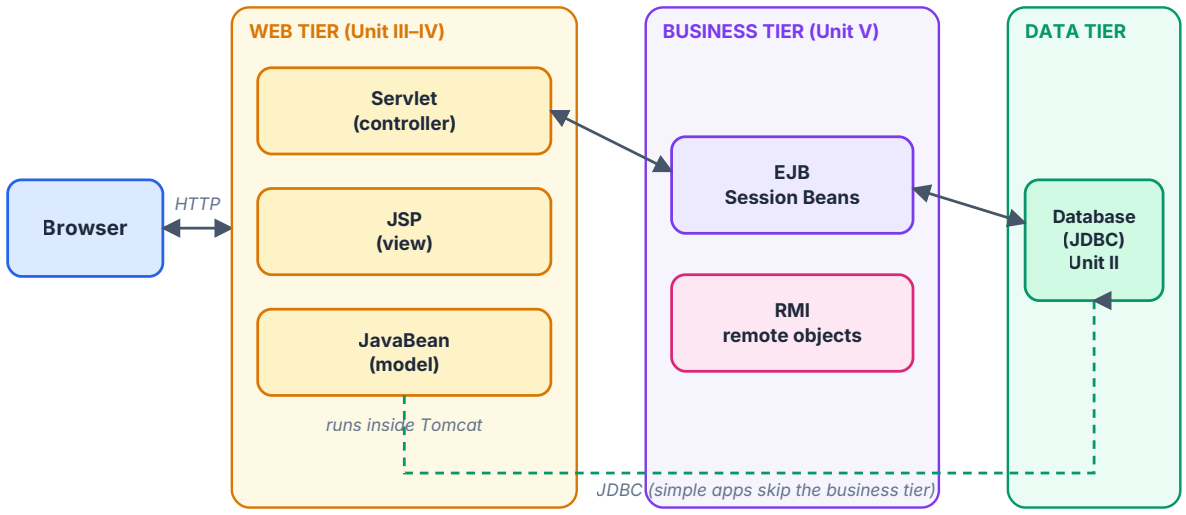


Fig. 1 — Three-tier Java enterprise application — Units II to V in one picture

🏭 Real-life use case — One request, end to end

A student opens a **college result portal**: the browser sends an HTTP request (Unit III) to a **Servlet** controller, which asks a **JavaBean/DAO** (Unit IV) for data; the DAO uses **JDBC** (Unit II) to query the database; the controller forwards the result to a **JSP** that renders HTML. In bigger systems the business logic lives in **EJB** or is reached over **RMI** (Unit V). Core Java (Unit I) is the language everything is written in.

Practice reading: GeeksforGeeks Java tutorial hub — geeksforgeeks.org/java · Oracle docs — docs.oracle.com/javase · Jakarta EE — jakarta.ee/specifications · Apache Tomcat — tomcat.apache.org

UNIT I · 8 HOURS

Review of Java Concepts

Features | OOP | JDK / JRE / JVM | Data types | Memory | Control flow | Call stack | Casting | Overloading | Strings | Arrays

1.1 Features of Java

Definition

Java is a high-level, object-oriented, platform-independent programming language designed to be simple, secure and robust.

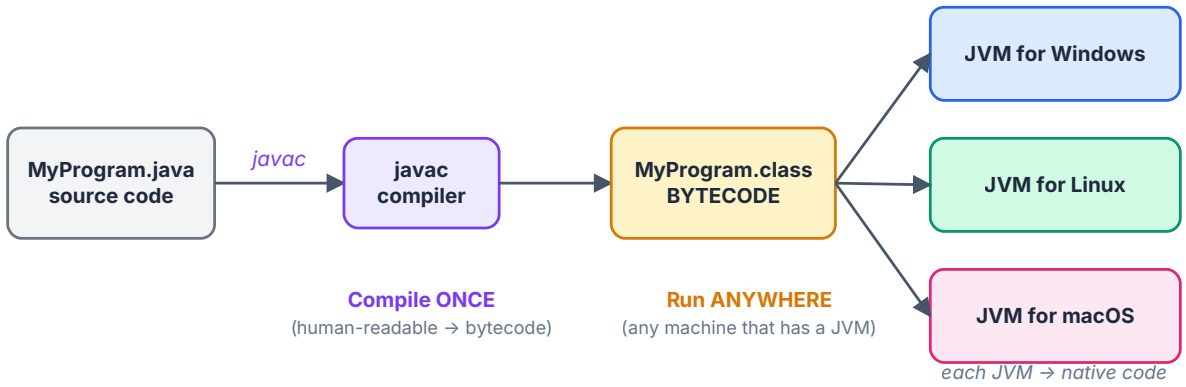


Fig. 2 — Write Once, Run Anywhere — one bytecode file, a different JVM per operating system

Feature	Explain like this in the exam
Simple	Removes complex C++ features — pointers, operator overloading, multiple inheritance of classes.
Object-oriented	Everything except the 8 primitive types is an object → reuse through inheritance, encapsulation, polymorphism.
Platform independent	Source → **bytecode** → any machine with a JVM. "Write Once, Run Anywhere (WORA)".
Secure	No explicit pointers, bytecode verifier, sandbox via ClassLoader / Security Manager.
Robust	Strong type checking + automatic garbage collection + mandatory exception handling.
Multithreaded	Built-in `Thread` class and `Runnable` interface for concurrent tasks.
Architecture neutral	Bytecode format does not depend on any CPU architecture.
Portable	Primitive sizes are fixed by the specification — `int` is always 32 bit.
High performance	JIT compiler turns hot bytecode into native machine code at runtime.
Distributed	`java.net` and RMI make network programs easy.
Dynamic	Classes are loaded on demand; reflection lets programs inspect classes at runtime.

🏭 **Real-life use case — Why companies choose Java**

A bank writes its back-end once on a Linux server, while the same compiled `.class` files run on a developer's Windows laptop and a test server on macOS — no recompilation. Android apps, banking cores, e-commerce back-ends (Flipkart-style order systems) and big-data tools like Hadoop rely on this portability plus automatic memory management.

⭐ **Exam tip**

For 5+ marks pick the top six — Simple, OOP, Platform independent, Robust, Secure, Multithreaded — and write 2 lines each.

📖 GFG: Features of Java

1.2 Object-Oriented Programming Overview

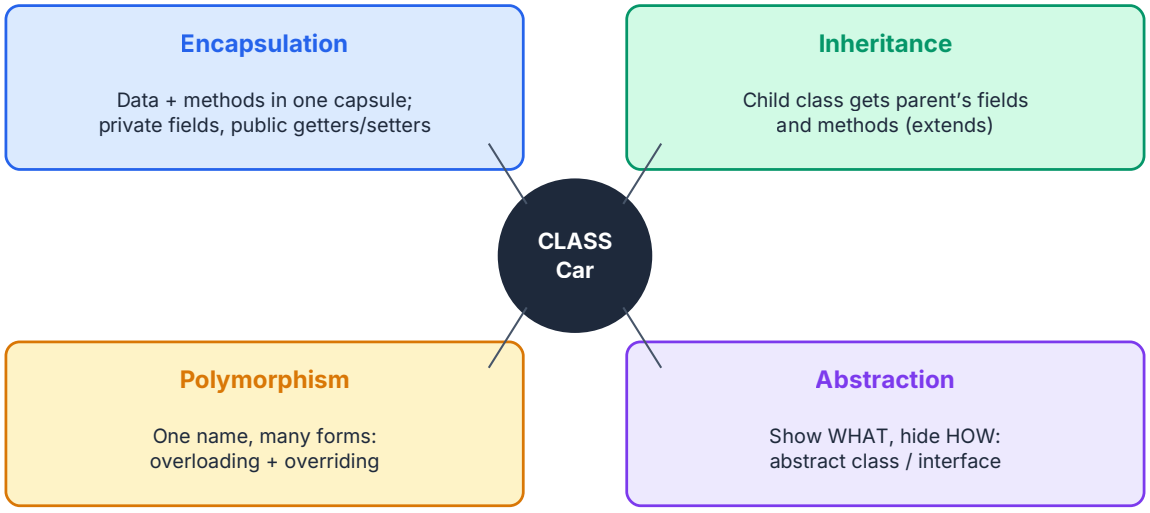


Fig. 3 — The four pillars of OOP around a Car class

1. **Encapsulation** — bind data (fields) and methods in one unit (class); hide data with `private` and expose it through getters/setters.
2. **Inheritance** — a subclass acquires properties and behaviour of a superclass → code reuse.
3. **Polymorphism** — many forms: *overloading* (compile-time) and *overriding* (runtime).
4. **Abstraction** — show only essentials, hide implementation; achieved with abstract classes and interfaces.

Java-specific OOP notes

- No free/global functions — every method belongs to a class (stricter than C++).
- Single inheritance for classes; multiple inheritance only through **interfaces**.
- Non-static, non-final, non-private methods are virtual by default → dynamic method dispatch.

🏭 Real-life use case — Banking example

`Account` class: balance is **private** (encapsulation); `SavingsAccount` and `CurrentAccount` **extend** it (inheritance); `calculateInterest()` behaves differently in each (polymorphism); the app only calls the `Account` interface and never sees the internal calculation (abstraction).

🧠 Yaad rakho

A-PIE = Abstraction, Polymorphism, Inheritance, Encapsulation — chaaron pillars ek saath yaad rahenge.

📖 GFG: OOPs Concepts in Java

1.3 Introduction to Java Technologies — JDK, JRE, JVM

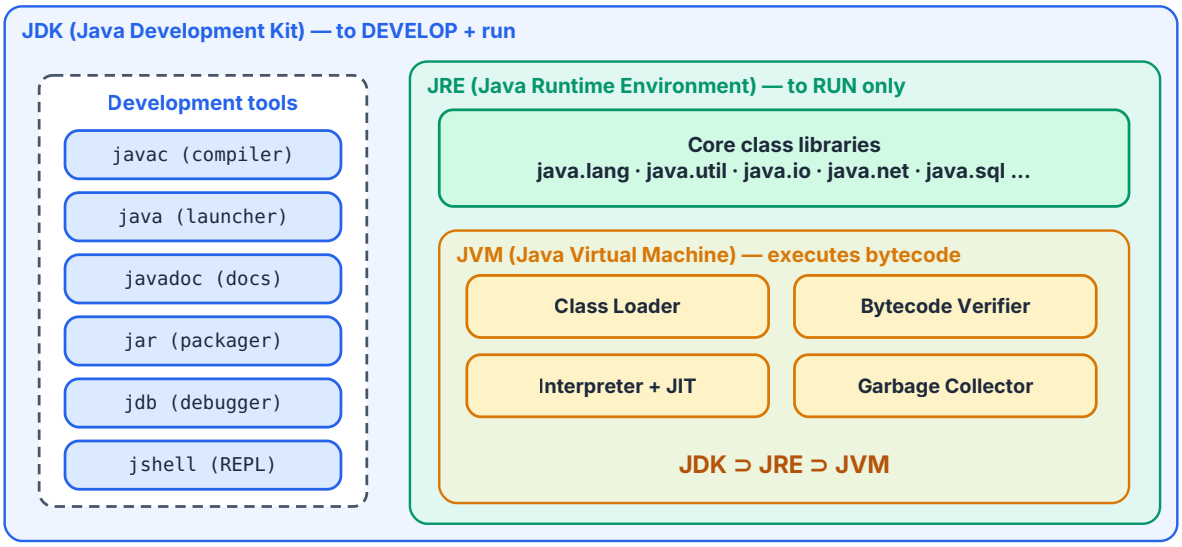


Fig. 4 — JDK contains the JRE, which contains the JVM

Term	Contains	Needed for
JDK	JRE + development tools (`javac` , `javadoc` , `jar` , `jdb` , `jshell`)	Develop *and* run
JRE	JVM + core class libraries	Run only
JVM	Class loader, verifier, interpreter + JIT, garbage collector	Executes bytecode (platform-specific)

Java editions

Edition	Purpose	Syllabus
Java SE	Core language, desktop / console apps	Units I–II
Jakarta EE (Java EE)	Enterprise APIs — Servlets, JSP, EJB	Units III–V
Java ME	Embedded / mobile (legacy; replaced by Android)	—

Compile and run: `javac` translates `MyProgram.java` into bytecode (`MyProgram.class`); the JVM interprets / JIT-compiles it to native instructions (see Fig. 2).

📖 GFG: JDK vs JRE vs JVM

1.4 Writing a Simple Java Program

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```bash
javac HelloWorld.java     # compiles to HelloWorld.class
java  HelloWorld          # JVM loads and runs it
```

Part of main()	Why
`public`	JVM (outside the class) must be able to call it
`static`	Called without creating an object first
`void`	Returns nothing to the JVM
`String[] args`	Receives command-line arguments

The public class name must match the file name. `System.out` is a `PrintStream` object for standard output.

> ⭐ **Exam tip**
>
> 2-mark favourite: "Explain every word of `public static void main(String[] args)` " — use the table above.

📖 GFG: main() method in Java

1.5 Data Types

> 📘 **Definition**
>
> Java has **primitive types** (8, built in, hold raw values) and **reference types** (objects, arrays, Strings — hold the address of data on the heap).

Type	Size	Default	Range
byte	1 byte	0	−128 to 127
short	2 bytes	0	−32,768 to 32,767
int	4 bytes	0	≈ −2.1 B to 2.1 B
long	8 bytes	0L	≈ $\pm 9.2 \times 10^{18}$
float	4 bytes	0.0f	≈ $\pm 3.4 \times 10^{38}$ (7 digits)
double	8 bytes	0.0d	≈ $\pm 1.8 \times 10^{308}$ (15 digits)
char	2 bytes	'\u0000'	0 to 65,535 (Unicode)
boolean	JVM-dependent	false	true / false

```java
int i = 100000;            long l = 10000000000L;     // note the L
float f = 3.14f;           double d = 3.14159;        // note the f
Integer boxed = 10;        // autoboxing: primitive -> object
int back = boxed;          // unboxing:   object -> primitive
```

Memory units quick reference

1 Byte = 8 bits · 1 KB = 1,024 B · 1 MB = 1,024 KB · 1 GB = 1,024 MB · 1 TB = 1,024 GB · 1 PB = 1,024 TB · 1 EB = 1,024 PB · 1 ZB = 1,024 EB · 1 YB = 1,024 ZB.

> 💡 **Why it matters**
>
> Collections such as `ArrayList` and `HashMap` can only store **objects**, not primitives — that is exactly why wrapper classes and autoboxing exist.

📖 GFG: Data Types in Java

1.6 Variables and Memory Concepts

Variable kind	Where stored	Copies	Lifetime
static (class)	Method area / Metaspace	ONE, shared	Until class unloads
instance	Heap (inside the object)	One per object	While object is reachable
local	Stack (method frame)	One per call	Only during the call; no default value

```java
public class MemoryDemo {
    static int classField = 10;      // shared by ALL objects
    int instanceField = 20;          // separate in every object
    void demo() { int localVar = 30; }   // must be initialised before use

    public static void main(String[] a) {
        MemoryDemo o1 = new MemoryDemo(), o2 = new MemoryDemo();
        o1.instanceField = 99;
        System.out.println(o2.instanceField);       // 20 - separate copy
        classField = 77;
        System.out.println(MemoryDemo.classField);  // 77 - shared
    }
}
```

JVM runtime memory areas

Each thread gets its own **Stack, PC register and Native method stack** (private); the **Heap** and **Method area** are shared by all threads — that is why instance / static data needs synchronisation but local variables do not.

> 🧠 **Yaad rakho**
>
> **Heap = objects, Stack = calls.** Jo `new` se bana wo heap me; method ke andar ka local variable stack me.

📖 GFG: JVM Memory Model in Java

1.7 Control Statements

```java
int marks = 75;
if (marks >= 90) System.out.println("A+");
else if (marks >= 75) System.out.println("A");
else System.out.println("B");

int day = 3;
switch (day) {                          // classic: break stops fall-through
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Other");
}
String name = switch (day) {            // Java 14+ arrow form, returns a value
    case 1 -> "Mon";
    case 2 -> "Tue";
    default -> "Other";
};
String result = (marks >= 40) ? "Pass" : "Fail";   // ternary
```

> ⭐ **Exam tip**
>
> In the classic `switch`, `break` prevents **fall-through**; the arrow form does not fall through and can directly produce a value.

📖 GFG: Decision Making in Java

1.8 Looping Constructs

Loop	Condition checked	Use when
`for`	Before each iteration	Iteration count known
`while`	Before body (may run 0 times)	Count unknown
`do-while`	After body (runs at least once)	Menu / input validation
for-each	Iterates every element	Arrays and collections, no index needed

```java
for (int i = 0; i < 5; i++) System.out.println(i);
int j = 0;  do { System.out.println(j); j++; } while (j < 5);
for (int n : new int[]{10, 20, 30}) System.out.println(n);

outer:
for (int a = 0; a < 3; a++) {
    for (int b = 0; b < 3; b++) {
        if (b == 1) continue outer;    // next iteration of OUTER loop
        if (a == 2) break outer;       // leave OUTER loop completely
        System.out.println(a + "," + b);
    }
}
```

📖 GFG: Loops in Java

1.9 Method Call Stack and Activation Records

> 📘 **Definition**
>
> Every method call creates a **stack frame (activation record)** on the calling thread's call stack that stores that call's local data.

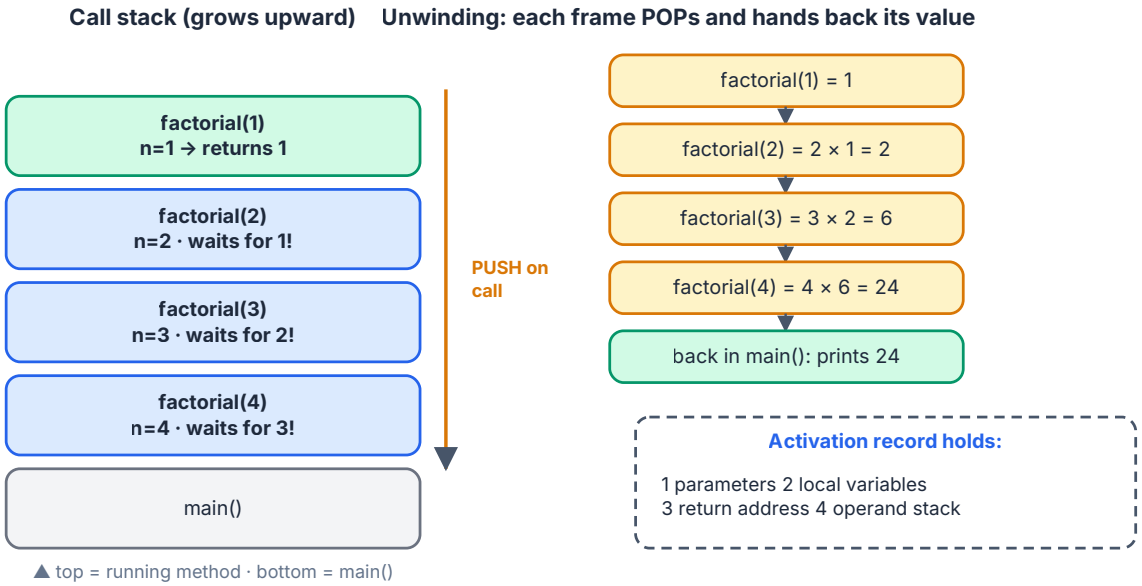


Fig. 5 — Computing factorial(4): frames are pushed on every call, then popped as results return

```java
static int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);     // each call pushes a NEW frame
}
```

An activation record stores four things: **parameters, local variables, return address** and the **operand stack**.

> ⚠️ **Careful**
>
> Recursion without a base case (or too deep) exhausts the fixed-size stack and throws `StackOverflowError`.

📖 GFG: Call Stack / Recursion in Java

1.10 Argument Promotion and Casting

> 📘 **Definition**
>
> **Widening** (implicit, small → large, safe) and **narrowing** (explicit cast, large → small, possible data loss) are the two kinds of type conversion.

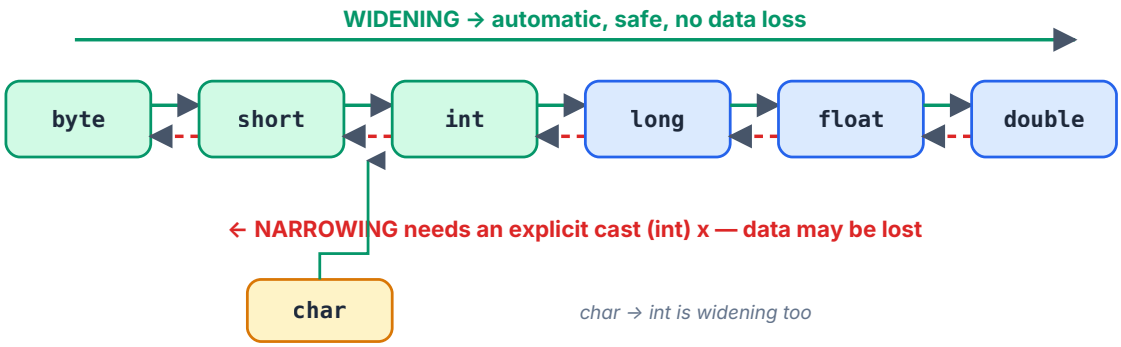


Fig. 6 — Widening is automatic; narrowing needs a cast

```java
byte b = 10;  int i = b;  long l = i;  double d = l;       // widening: automatic

double pi = 3.99;  int t = (int) pi;                        // 3 - fraction lost
long big = 130L;   byte o = (byte) big;                     // overflow, wraps around

byte x = 5, y = 10;
int sum = x + y;              // byte + byte is promoted to int
// byte wrong = x + y;        // compile error - needs (byte)(x + y)

void show(long n) { }         show(5);   // int 5 widened to long automatically
```

💡 **Why it matters**

Widening is always safe because the target type can hold every value of the source. Narrowing may lose information, so the compiler forces you to acknowledge the risk with a cast.

📖 GFG: Type Conversion in Java

1.11 Scope of Declaration and Method Overloading

```java
public class ScopeDemo {
    int field = 100;                        // class scope
    void method1() {
        int local = 10;                     // method scope
        { int blockVar = 20; }              // block scope - gone after }
    }
    void print(int i)        { }            // overloading: same name,
    void print(double d)     { }            // different parameter lists
    void print(String s)     { }
    void print(int a, int b) { }
}
```

📘 **Definition**

Method overloading = several methods in one class with the same name but a different parameter list (number, type or order). Return type alone is **not** enough.

Overload resolution order (decided at compile time)

1. Exact type match

2. Widening primitive conversion
3. Autoboxing / unboxing
4. Varargs

The compiler tries the steps in this order and stops at the first one that finds a matching method.

📖 GFG: Method Overloading in Java

1.12 String Handling

String constructors and the String Pool

```java
String s1 = "Java";                          // literal -> String Pool
String s2 = new String("Java");              // new heap object, NOT pooled
String s3 = new String(new char[]{'J','a','v','a'});
String s5 = new String(new byte[]{72, 105}); // "Hi"
System.out.println(s1 == s2);                // false - different objects
System.out.println(s1.equals(s2));           // true  - same content
```

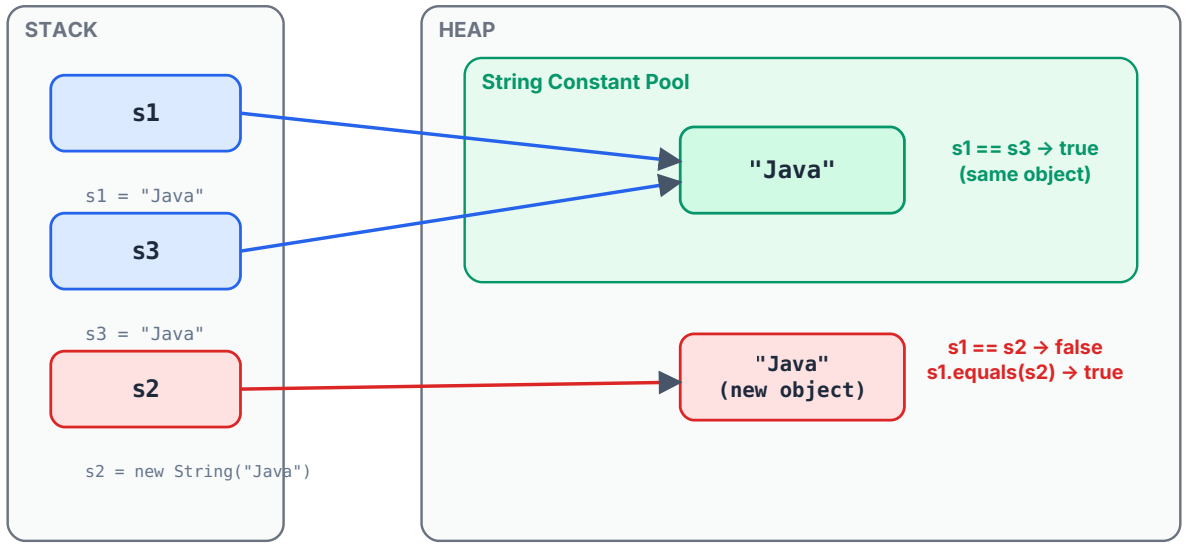


Fig. 7 — Literals share one object in the pool; new String(...) always creates another

The **String Constant Pool** is a special area inside the heap. For a literal, the JVM checks the pool first and reuses an existing object; `new String(...)` bypasses the pool.

Operators, extraction and comparison

```java
String c = "Hello" + " " + "World";   c += "!";      // + is Java's only overloaded operator
String x = "Hi";  x.concat(" there");                // result discarded - x is still "Hi"

String str = "Java Programming";
str.charAt(0);  str.substring(5);  str.substring(0, 4);  str.length();
str.toCharArray();  str.getBytes();  str.getChars(0, 4, buf, 0);

"apple".equals("Apple");              // false (case-sensitive)
"apple".equalsIgnoreCase("Apple");    // true
"apple".compareTo("Apple");           // > 0 (lowercase > uppercase in Unicode)
"apple".startsWith("app");  "apple".endsWith("le");  "apple".contains("ppl");
```

⚠️ **Careful**

Use `.equals()` to compare string **content**. `==` compares references (memory addresses).

Why String is immutable

- **Security** — paths, URLs and class names cannot be changed after validation.
- **Pool efficiency** — sharing literals is only safe if strings never change.
- **Thread safety** — immutable objects can be shared without synchronisation.
- **Hash caching** — hash code computed once; ideal for `HashMap` keys.

String vs StringBuffer vs StringBuilder

String	StringBuffer	StringBuilder
Immutable	Mutable	Mutable
Thread-safe (immutable)	Thread-safe (synchronized)	NOT thread-safe
Slow for repeated changes (new object each time)	Slower (locking overhead)	Fastest for single thread

```java
JAVA
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World").insert(5, ",").reverse().reverse();   // Hello, World
sb.replace(0, 5, "Hi").delete(0, 3).setCharAt(0, 'w');    // methods change the SAME object
```

🏭 **Real-life use case — Where this matters**

Building a big report or SQL string in a loop with `+` creates thousands of temporary String objects; a `StringBuilder` does it in one buffer. Web servers (single request thread) use `StringBuilder`; legacy shared-buffer code uses `StringBuffer`.

⭐ **Exam tip**

The String / StringBuffer / StringBuilder table is asked almost every year — memorise it exactly.

📖 GFG: String vs StringBuffer vs StringBuilder

1.13 Arrays

📘 **Definition**

An array is a fixed-size, ordered collection of elements of the **same type**, stored contiguously in memory and treated as an **object** in Java.

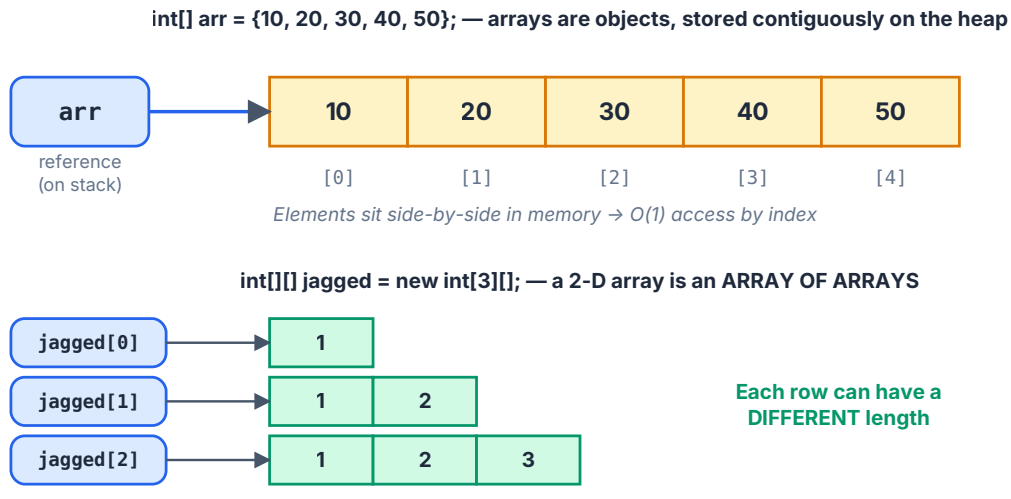


Fig. 8 — Array in memory, and a jagged 2-D array (an array of arrays)

```java
int[] a1 = new int[5];                // all zeros
int[] a2 = {1, 2, 3, 4, 5};           // shorthand, only at declaration
for (int n : a2) System.out.println(n);   // enhanced for: read-only view

static void modify(int[] arr) { arr[0] = 999; }   // caller's array IS changed
int[][] jagged = new int[3][];  jagged[0] = new int[]{1};  jagged[1] = new int[]{1, 2};
```

💡 Why it matters — Pass by value, always

Java is always **pass by value**. For objects and arrays the value copied is the **reference** — so changes made through it inside the method affect the original object, but re-assigning the parameter does not.

Variable-length arguments (varargs) and command-line arguments

```java
static int sum(int... numbers) {           // zero or more ints, seen as int[]
    int total = 0;  for (int n : numbers) total += n;  return total;
}
static void describe(String name, int... scores) { }   // varargs must be LAST
// main(String[] args):   java CmdArgsDemo hello world 123   ->  args.length == 3
```

Rule: at most one varargs parameter, and it must be last so the compiler knows where the fixed parameters end.

🏭 Real-life use case

A marks table stored as a jagged array: each student has a different number of subjects. A reusable `average(int... marks)` method works for 3 or 6 subjects without overloads.

📖 GFG: Arrays in Java

❓ Likely exam questions — Unit I

- Explain the features of Java. Differentiate JDK, JRE and JVM with a diagram.
- Explain the JVM memory areas / draw the call stack for a recursive factorial.
- Widening vs narrowing conversion — with examples.
- Why is String immutable? Compare String, StringBuffer and StringBuilder.
- What is method overloading? State the overload resolution order.
- What are varargs? Rules for using them.

UNIT II · 8 HOURS

Inheritance, Packages, Exceptions, Threads, Applets & JDBC

Inheritance | Packages | Interfaces | Exception handling | Multithreading | Synchronization | Applets | JDBC

2.1 Inheritance: Extending Classes

Definition

Inheritance lets a subclass acquire the fields and methods of a superclass using `extends`, giving code reuse and an **"is-a"** relationship.

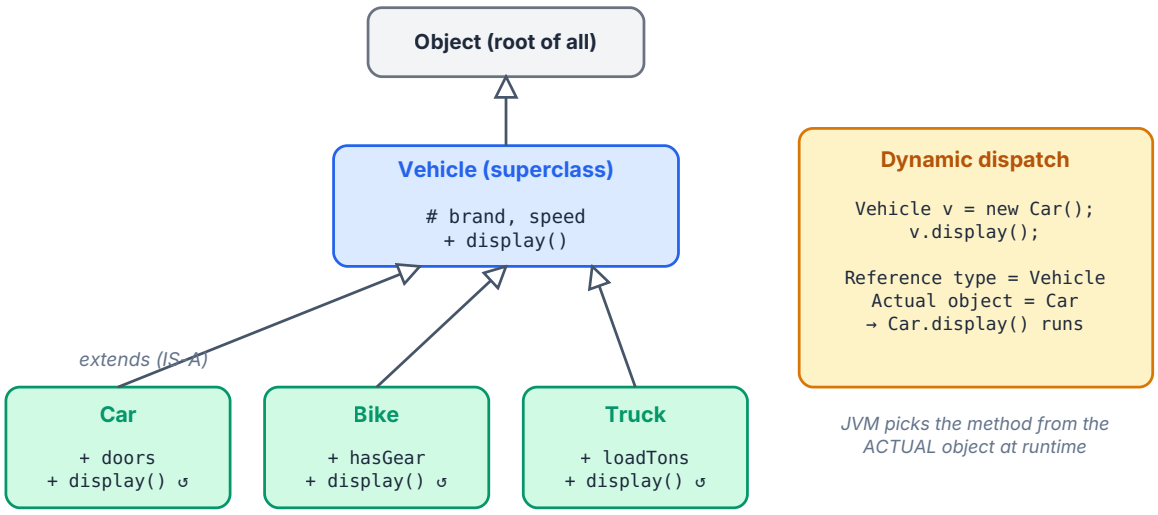


Fig. 9 — Class hierarchy (UML style) and dynamic method dispatch

```java
class Vehicle {
    protected String brand;  protected int speed;
    Vehicle(String brand, int speed) { this.brand = brand; this.speed = speed; }
    void display() { System.out.println(brand + " @ " + speed + " km/h"); }
}
class Car extends Vehicle {
    int doors;
    Car(String brand, int speed, int doors) {
        super(brand, speed);                // must be the FIRST statement
        this.doors = doors;
    }
    @Override void display() { super.display(); System.out.println("Doors: " + doors); }
}
Vehicle v = new Car("Tata", 180, 4);          // upcasting: always safe
v.display();                                  // Car's version runs (dynamic dispatch)
if (v instanceof Car c2) System.out.println(c2.doors);   // safe downcast
```

Rules of inheritance

- Single inheritance for classes; a class can **implement many interfaces**.
- `super(...)` must be the first statement of a child constructor; `super.method()` calls the parent's version.
- Every class implicitly extends `Object` (gets `toString()`, `equals()`, `hashCode()`).
- A `final` class cannot be extended; a `final` method cannot be overridden.
- Upcasting is implicit and safe; downcasting needs a cast and an `instanceof` check to avoid `ClassCastException`.
- **Dynamic dispatch:** the JVM chooses the method from the *actual object* at runtime, not the reference type.

🏭 Real-life use case — Where you see it

Payroll software: `Employee` holds name, id and `calculateSalary()`; `Manager`, `Developer` and `Intern` extend it and override only the salary rule. A loop over `List<Employee>` just calls `calculateSalary()` and every object does the right thing — no `if/else` on type.

📖 GFG: Inheritance in Java

2.2 Packages and Interfaces

📘 Definition

A **package** is a namespace / folder mechanism that groups related classes and interfaces, avoids name clashes and controls access.

```java
// File: com/rohit/utils/MathHelper.java   (folders MUST mirror the package name)
package com.rohit.utils;
public class MathHelper { public static int square(int x) { return x * x; } }

// Using it
import com.rohit.utils.MathHelper;        // single class
import java.util.*;                       // all classes in that package (not sub-packages)
import static java.lang.Math.PI;          // static import: use PI directly
```

```bash
javac -d build src/com/rohit/utils/MathHelper.java    # -d places .class files per package
java  -cp build com.rohit.utils.Main                   # -cp = CLASSPATH
# CLASSPATH tells the JVM / compiler where to find .class files and JARs.
# Set by: environment variable CLASSPATH, or -cp flag. Default: current directory (.)
```

Access protection

Modifier	Same class	Same package	Subclass (other package)	Other package
`private`	✓	✗	✗	✗
(default)	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

🧠 **Yaad rakho**

Chhote se bade: **private < default < protected < public** — har step pe ek naya group access paata hai.

Creating your own package — full example

```java
// shapes/Shape.java
package shapes;   public interface Shape { double area(); }
// shapes/Circle.java
package shapes;
public class Circle implements Shape {
    private double r;  public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}
// Main.java
import shapes.*;
public class Main { public static void main(String[] a) { System.out.println(new Circle(5).area()); } }
```

Interface vs abstract class

Abstract class	Interface
Cannot support multiple inheritance (one parent)	Supports multiple inheritance (a class implements many)
Can have a constructor	Cannot have a constructor
Can have methods with a body	Mainly method signatures (default / static allowed since Java 8)
Can have static and private methods	Only limited static / private helpers (Java 8/9+)

🏭 **Real-life use case**

A payment app defines `interface PaymentMethod` (with `pay(double amt)`). `UpiPayment`, `CardPayment` and `WalletPayment` implement it. Checkout code depends only on the interface, so a new payment type is added without touching it.

📖 GFG: Packages · Interfaces in Java

2.3 Exception Handling

Definition

Exception handling detects and handles runtime errors so the normal program flow can continue, using `try`, `catch`, `finally`, `throw` and `throws`.

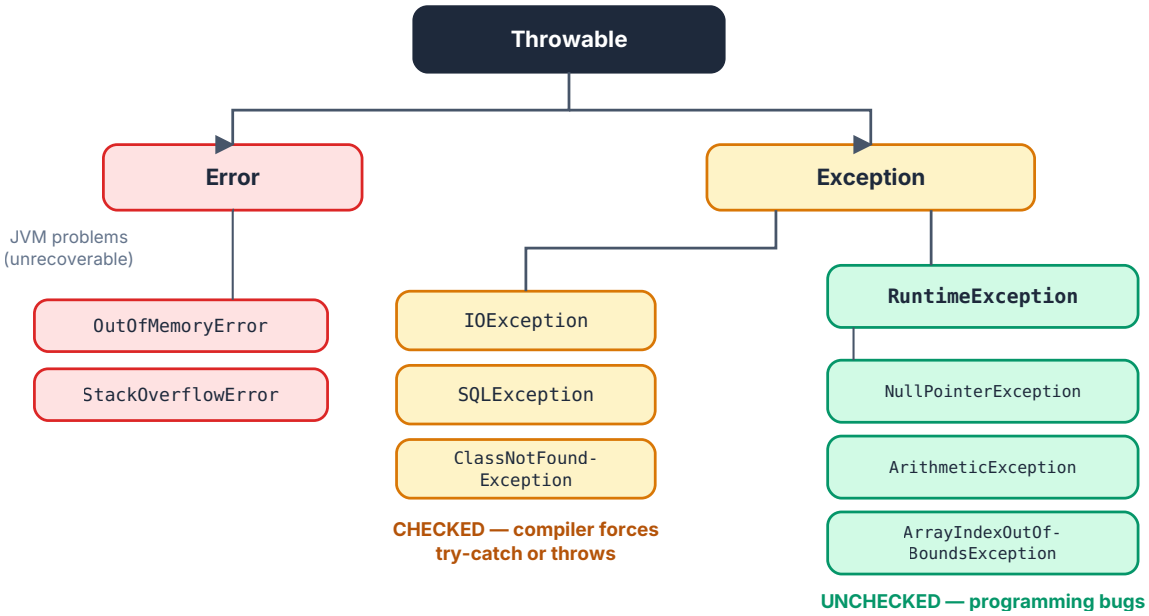


Fig. 10 — Throwable hierarchy — Error, checked and unchecked exceptions

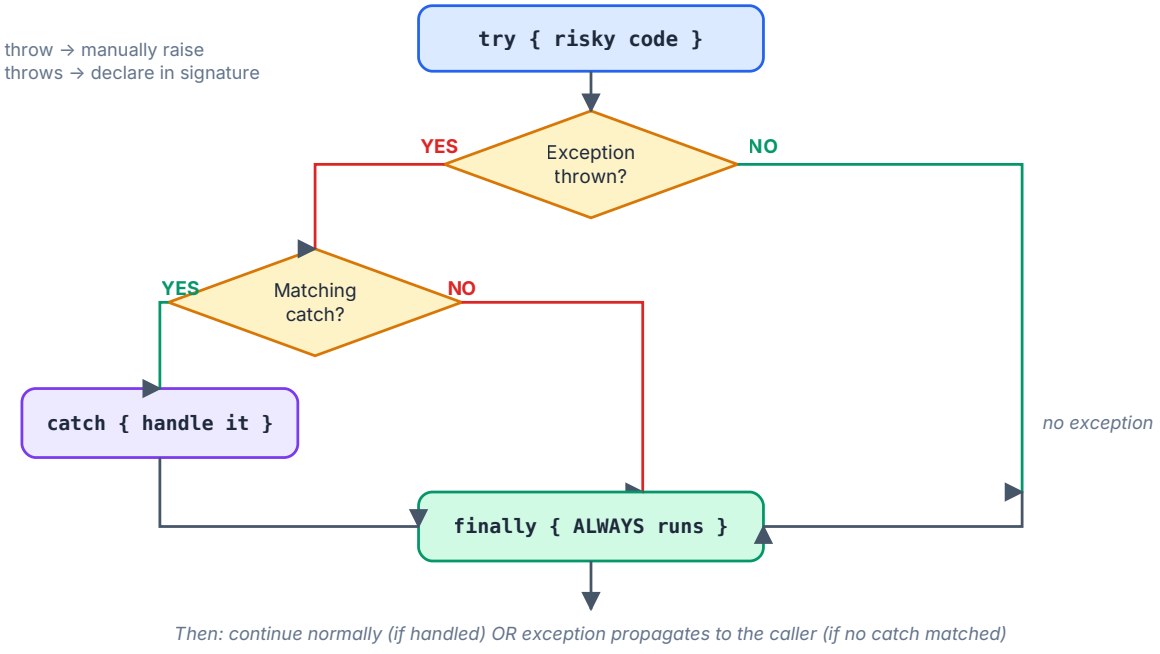


Fig. 11 — Execution flow of try-catch-finally

Keyword	Meaning
`try`	Block of code monitored for exceptions
`catch`	Handles a specific exception type thrown from try
`finally`	ALWAYS runs — cleanup (close files / connections)
`throw`	Explicitly raises an exception object
`throws`	Declares checked exceptions a method may pass to its caller

```java
JAVA
static int divide(int a, int b) {
    if (b == 0) throw new ArithmeticException("Cannot divide by zero");
    return a / b;
}
try {
    System.out.println(divide(5, 0));
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Cleanup always runs");
}

class InvalidAgeException extends Exception {          // custom CHECKED exception
    InvalidAgeException(String msg) { super(msg); }
}
static void validateAge(int age) throws InvalidAgeException {
    if (age < 18) throw new InvalidAgeException("Must be 18+");
}
```

Checked vs unchecked

Checked	Unchecked (Runtime)
Verified by the compiler	Not checked at compile time; occurs at runtime
Must be caught or declared with `throws`	No such requirement
Extends `Exception` (not RuntimeException)	Extends `RuntimeException`
Recoverable conditions: `IOException`, `SQLException`	Programming bugs: `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`

💡 Why it matters — When to use exceptions

Use exceptions only for **exceptional** conditions — missing file, failed network call, invalid input — never as a replacement for normal control flow such as breaking out of a loop.

🏭 Real-life use case

A file-upload feature: `try` opens the file and saves it to disk; `catch (IOException e)` shows "upload failed, try again"; `finally` always closes the stream so the server never leaks file handles — even when the upload crashes midway.

📖 GFG: Exception Handling in Java

2.4 Multithreading

📘 **Definition**

A **thread** is the smallest unit of CPU execution inside a process. **Multithreading** runs several threads concurrently; all threads of a process share the same memory (unlike separate processes).

The Java thread model — two ways to create a thread

```java
class MyThread extends Thread {                       // Method 1: extend Thread
    public void run() { System.out.println(getName() + " running"); }
}
class MyTask implements Runnable {                    // Method 2: implement Runnable (preferred)
    public void run() { System.out.println(Thread.currentThread().getName()); }
}
new MyThread().start();                               // NEVER call run() directly
new Thread(new MyTask()).start();
new Thread(() -> System.out.println("Lambda thread")).start();   // Runnable is functional
t1.join();                                            // wait for t1 to finish
```

extends Thread	implements Runnable (preferred)
Simple, but the class cannot extend anything else	Class stays free to extend another class
Task and thread mechanism tied together	Separates *what* to run from *how* it runs

⚠️ **Careful**

Always call `start()`, never `run()`. Calling `run()` directly is an ordinary method call on the current thread — no new thread is created.

Thread priorities

`setPriority(Thread.MIN_PRIORITY)` = 1, `NORM_PRIORITY` = 5 (default), `MAX_PRIORITY` = 10. A priority is only a **hint** to the scheduler — behaviour depends on the JVM / OS.

Thread life cycle

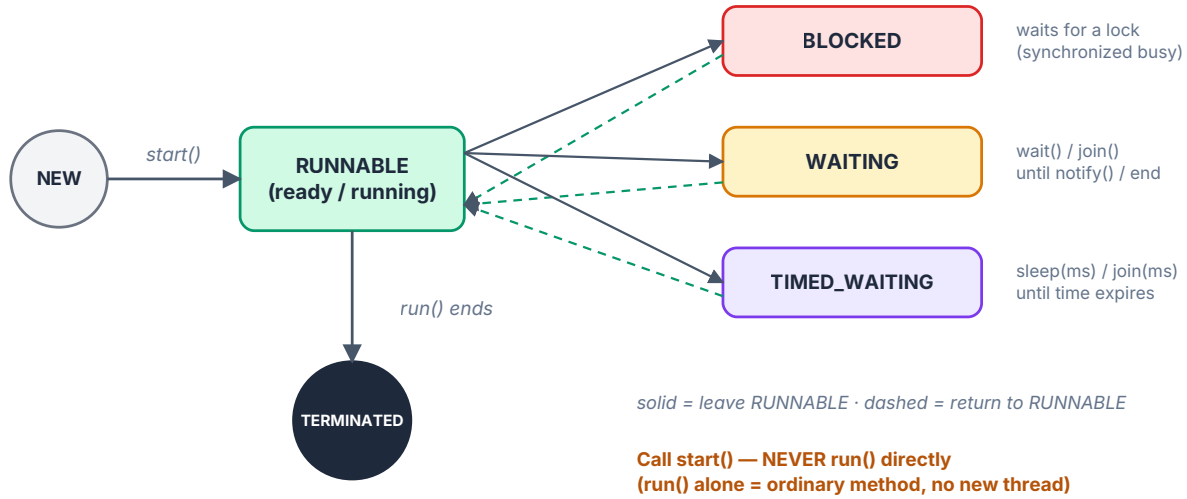


Fig. 12 — Thread states and the transitions between them

State	Meaning
NEW	Thread object created, `start()` not called yet
RUNNABLE	Executing, or ready and waiting for the scheduler
BLOCKED	Waiting to acquire a monitor lock (`synchronized`)
WAITING	Waiting indefinitely: `wait()` , `join()`
TIMED_WAITING	Waiting for a fixed time: `sleep(ms)` , `join(ms)`
TERMINATED	`run()` has completed

Thread synchronization

📘 **Definition**

Synchronization controls access of multiple threads to a shared resource and prevents a **race condition** — an inconsistent result caused by threads interleaving unpredictably.

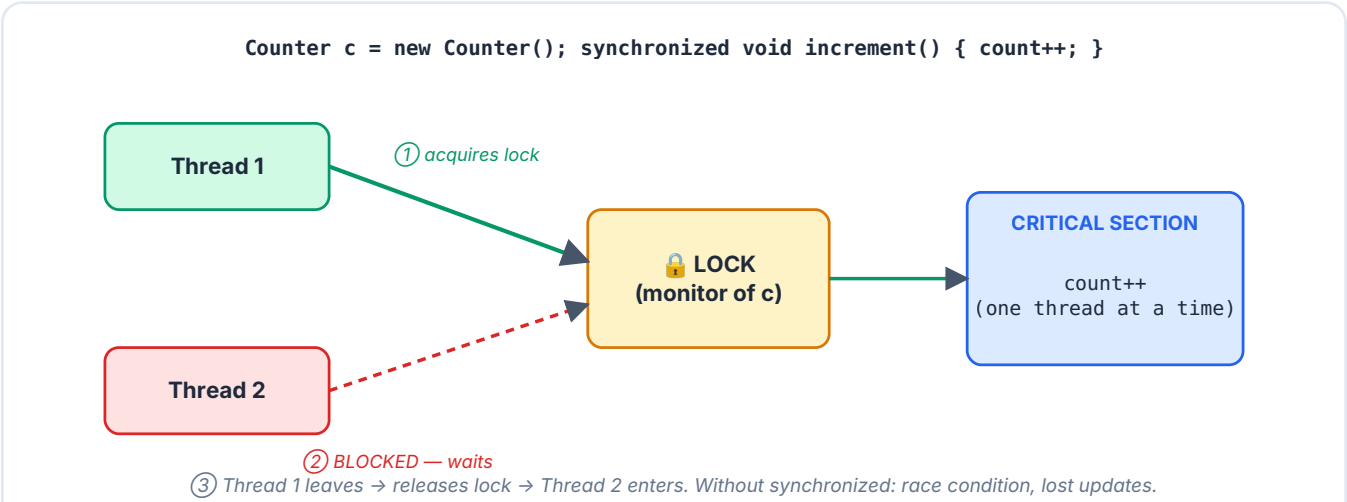


Fig. 13 — Only the thread holding the object's lock can enter the critical section

```java
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }     // one thread at a time per object
    public int getCount() { return count; }
}
// Two threads x 1000 increments  ->  always 2000 with synchronized

void transfer(Account from, Account to, double amt) {
    synchronized (from) {                                  // lock only the critical section
        synchronized (to) { from.debit(amt); to.credit(amt); }
    }
}
```

Every object has an intrinsic **lock (monitor)**. A synchronized *block* is usually better than a synchronized *method* because it locks only the minimum code, so more work runs concurrently.

🏭 Real-life use case

Railway ticket booking: two users click "Book" for the last seat at the same instant. Without synchronization both can read "1 seat left" and both book it (race condition). With a synchronized booking method, the second thread waits, then sees 0 seats. Other uses: web servers handle every request on its own thread; a download manager downloads many files in parallel; a UI keeps responding while a background thread loads data.

⭐ Exam tip

Draw the life-cycle figure together with the state table — 5+ mark question. Also remember: extends vs implements, start vs run, synchronized method vs block.

📖 GFG: Multithreading in Java

2.5 Applets

⚠️ Careful — Legacy topic

Applets were deprecated in Java 9 and removed in Java 17+; browsers dropped Java plugin support. They are covered here for syllabus / legacy exam purposes only.

📘 Definition

An **applet** is a small Java program that runs inside a web browser (or applet viewer), embedded in an HTML page.

```java
import java.applet.Applet;  import java.awt.Graphics;
public class HelloApplet extends Applet {
    public void paint(Graphics g) { g.drawString("Hello, Applet World!", 20, 20); }
}
```

```html
<applet code="HelloApplet.class" width="300" height="100"></applet>
```

Architecture: an applet runs inside the browser's JVM in a restricted **sandbox** — no local file access, no arbitrary network connections (only back to its origin server), no native code. A standalone application has full system access.

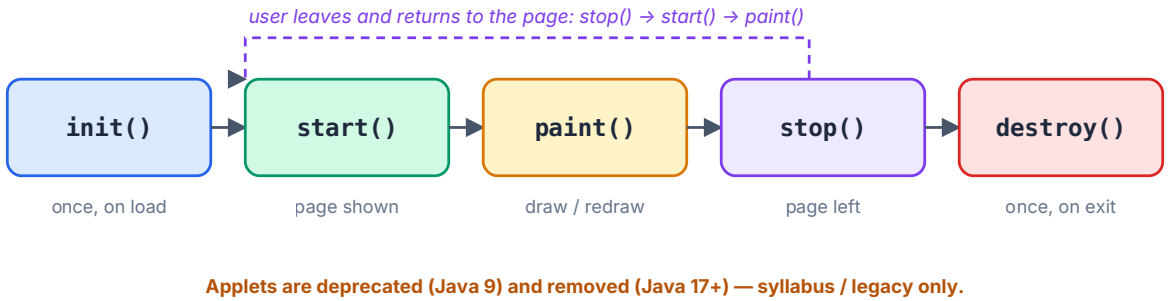


Fig. 14 — Applet life cycle methods

⭐ Exam tip

Order to write: first load `init()` → `start()` → `paint()`; leave and return `stop()` → `start()` → `paint()`; browser closes `stop()` → `destroy()`.

📖 GFG: Applet Life Cycle in Java

2.6 Database Connectivity: JDBC

📘 Definition

JDBC (Java Database Connectivity) is a Java API that gives a standard, vendor-independent way to connect to and work with relational databases.

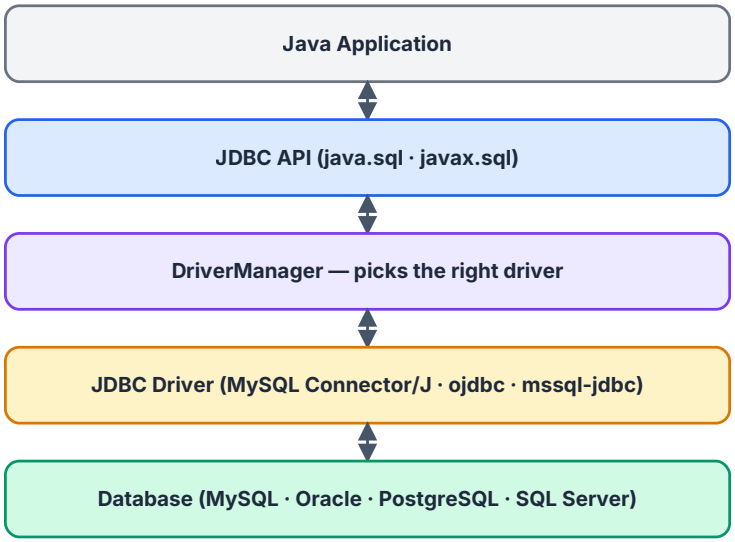


Fig. 15 — JDBC layers — the application never talks to the database directly

The four driver types

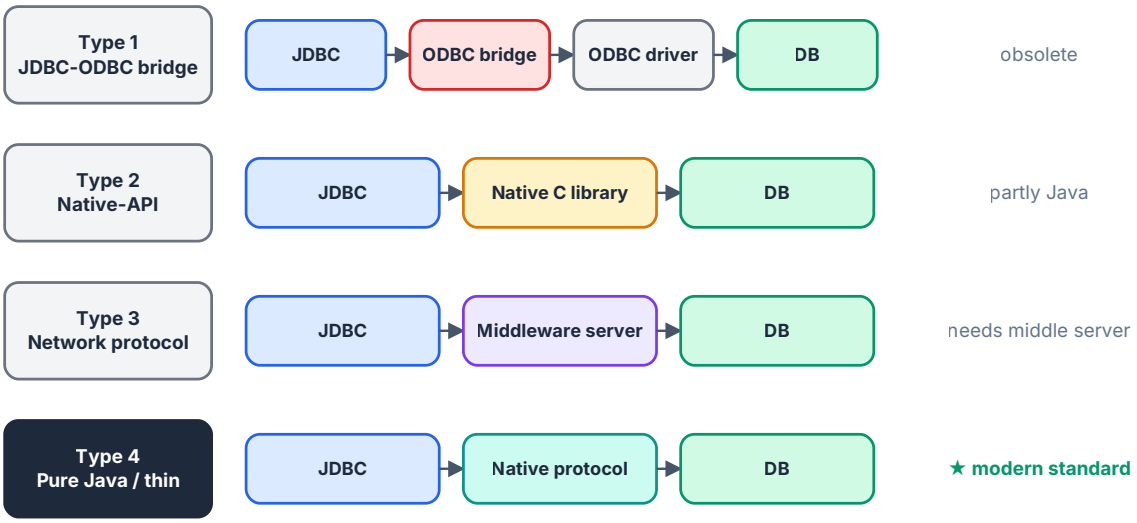


Fig. 16 — Type 1 to Type 4 JDBC drivers

- **Type 1** — JDBC-ODBC bridge; obsolete, removed from modern JDK.
- **Type 2** — Native-API driver; calls vendor C/C++ libraries; partly Java.
- **Type 3** — Network-protocol driver; talks to a middleware server; fully Java.
- **Type 4** — Pure Java "thin" driver; direct network protocol, no native code, no middleware — the modern standard (e.g. `mysql-connector-j`).

Steps to connect



try-with-resources closes Connection / Statement / ResultSet automatically

Fig. 17 — The standard JDBC workflow

```java
import java.sql.*;
String url = "jdbc:mysql://localhost:3306/schooldb";
try (Connection conn = DriverManager.getConnection(url, "root", "password");
     Statement stmt = conn.createStatement()) {

    ResultSet rs = stmt.executeQuery("SELECT id, name, cgpa FROM students");
    while (rs.next())
        System.out.println(rs.getInt("id") + " | " + rs.getString("name") + " | " + rs.getFloat("cgpa"));

    try (PreparedStatement ps = conn.prepareStatement("INSERT INTO students (name, cgpa) VALUES (?, ?)"))
{
        ps.setString(1, "Rohit");  ps.setFloat(2, 9.2f);
        ps.executeUpdate();                       // INSERT / UPDATE / DELETE
    }
} catch (SQLException e) { e.printStackTrace(); }
```

1. Load / register the JDBC driver (modern drivers auto-register via ServiceLoader).
2. Establish a connection — `DriverManager.getConnection(url, user, password)`.
3. Create a `Statement` / `PreparedStatement` / `CallableStatement`.
4. Execute — `executeQuery()` for SELECT, `executeUpdate()` for INSERT / UPDATE / DELETE.
5. Process the `ResultSet` and close the connection (try-with-resources does it automatically).

🏭 Real-life use case

An online **student-result portal**: the web app uses JDBC to read marks, insert new registrations and call a stored procedure (`CallableStatement`) that calculates ranks. Production apps add **connection pooling** (`DataSource`) so every request reuses ready connections instead of opening new ones, and **batch updates** to insert thousands of rows quickly.

📖 GFG: JDBC in Java

❓ Likely exam questions — Unit II

- Explain inheritance and the rules of `super` ; what is dynamic method dispatch?
- Access-modifier table; interface vs abstract class.
- Checked vs unchecked exceptions; explain try-catch-finally with a flow diagram.
- Thread creation methods, life cycle diagram, synchronization with an example.
- Applet life cycle (legacy question).
- JDBC driver types and the steps to connect; Statement vs PreparedStatement.

UNIT III · 8 HOURS

HTTP, Web Servers & Servlets

HTTP | Web vs application server | Tomcat setup | Servlet life cycle | GenericServlet vs HttpServlet | Forms | Session tracking | web.xml

3.1 Introduction to HTTP, Web Servers and Application Servers

Definition

HTTP (HyperText Transfer Protocol) is a **stateless**, text-based request / response protocol used between a client (browser) and a server over the web.

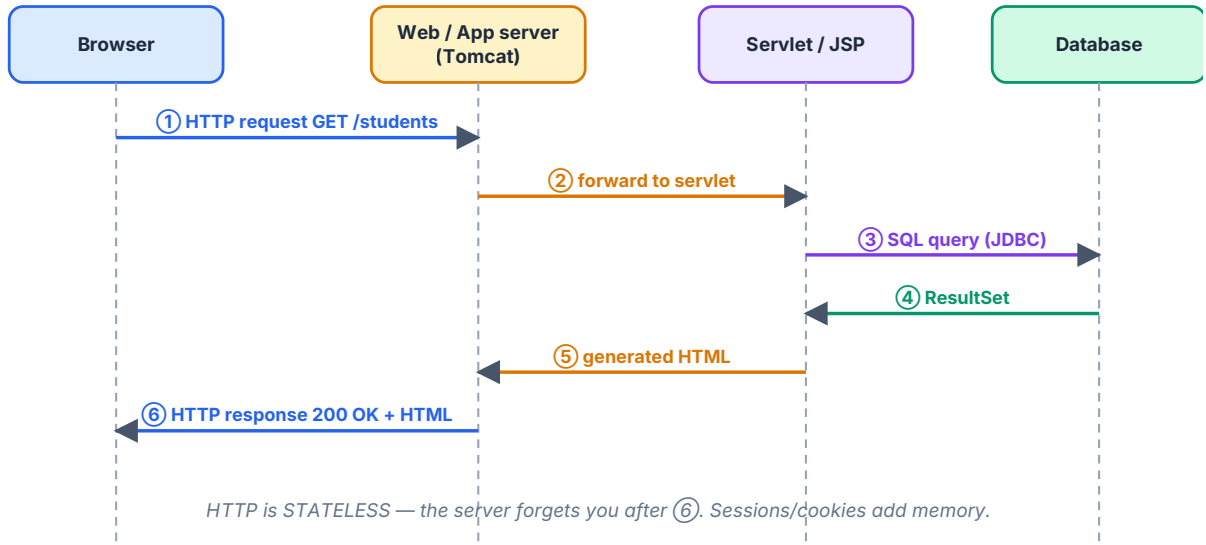


Fig. 18 — One HTTP request travelling through a Java web application and back

Why it matters — "Stateless" explained

Each HTTP request is independent — the server does not automatically remember anything from your previous request. That is why Java web technology needs **session tracking** (cookies, `HttpSession`, URL rewriting — Section 3.5).

Web server vs application server

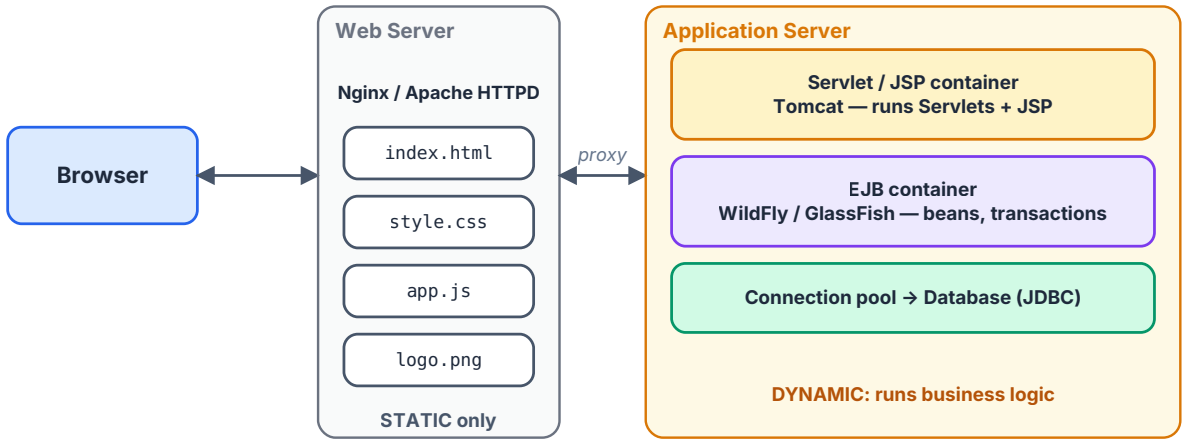


Fig. 19 — Static content from a web server, dynamic Java from an application server

Web server	Application server
Serves static content (HTML, CSS, JS, images)	Runs dynamic Java code (Servlets, JSP, EJB)
Examples: Apache HTTPD, Nginx	Examples: Apache Tomcat, WildFly, GlassFish
Cannot execute business logic	Executes business logic, connects to databases, manages transactions

Technically Tomcat is a lightweight **Servlet / JSP container**: it sits between a plain web server and a full application server (WildFly / GlassFish), which also supports EJB.

Real-life use case — Typical production set-up

Nginx faces the internet, serves images / CSS directly and **proxies** only dynamic URLs (e.g. `/login`, `/api/*`) to Tomcat. This keeps static traffic fast and lets Tomcat spend its threads on Java code.

Installing an application server (Apache Tomcat)

1. Download Tomcat from tomcat.apache.org and extract it.
2. Set `JAVA_HOME` (Tomcat needs a JDK).
3. Start: `bin/startup.sh` (Linux / Mac) or `startup.bat` (Windows).
4. Open `http://localhost:8080` to check; stop with `shutdown.sh`.

Folder / file	Purpose
`conf/server.xml`	Ports, connectors, hosts
`conf/web.xml`	DEFAULT deployment descriptor for all apps
`conf/context.xml`	Per-application context settings
`conf/tomcat-users.xml`	Manager / admin credentials
`webapps/`	Deployed apps (.war or exploded folders)
`lib/` · `logs/`	Shared JARs · server and app logs

📖 GFG: Web Server vs Application Server

3.2 Java Servlets

📘 Definition

A **Servlet** is a Java class that runs inside a servlet container (e.g. Tomcat) and dynamically handles HTTP requests to produce responses. It replaced CGI scripts and is faster because it is loaded **once** and each request runs on its own **thread**.

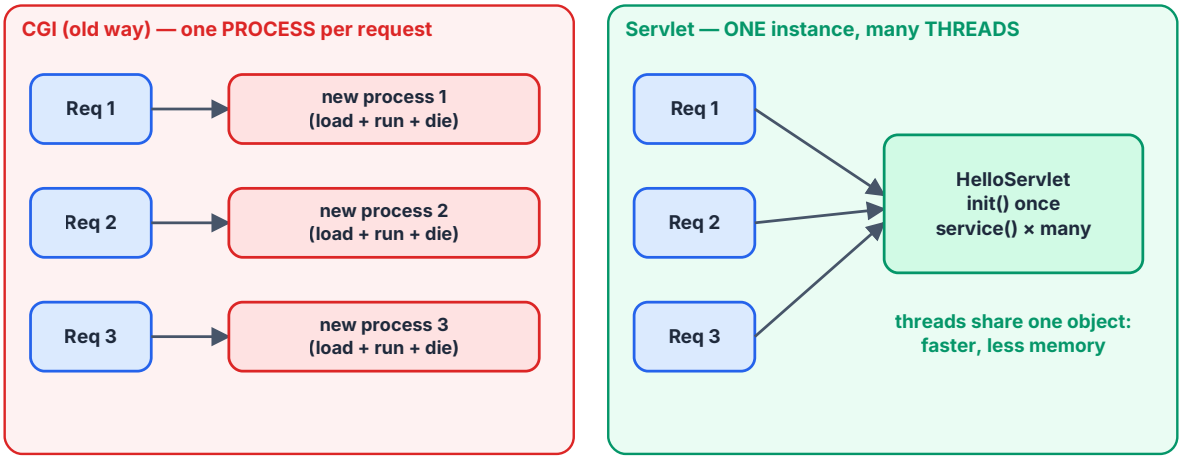


Fig. 20 — Why servlets beat CGI — threads instead of a new process per request

Servlet development process

1. Write a class extending `HttpServlet` (or implementing `Servlet`).
2. Override `doGet()` / `doPost()`.
3. Compile against the Servlet API JAR (provided by the container).
4. Package into a **WAR** with the correct directory structure.
5. Configure with `web.xml` or the `@WebServlet` annotation.
6. Deploy the `.war` into `webapps/`.
7. Container loads, initialises and routes requests to it.

```java
import jakarta.servlet.http.*;  import jakarta.servlet.annotation.WebServlet;  import java.io.*;

@WebServlet("/hello")                                   // Servlet 3.0+: no web.xml entry needed
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h1>Hello from Servlet!</h1>");
    }
}
```

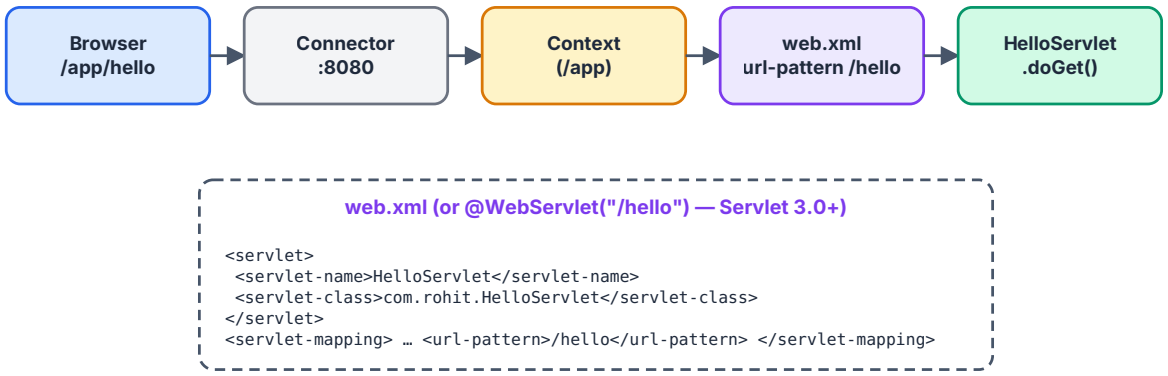


Fig. 21 — How a URL finds its servlet

GenericServlet vs HttpServlet

GenericServlet	HttpServlet
Protocol-independent	HTTP-specific
You override the single `service()` method yourself	`service()` is already implemented and dispatches to `doGet()` / `doPost()` / `doPut()` / `doDelete()` by HTTP method
Rarely used directly	Used in almost every real web application

Servlet life cycle

> ⭐ **Exam tip**
>
> Asked almost every exam. Write: `init()` once → `service()` for every request (dispatching to doGet / doPost) → `destroy()` once.

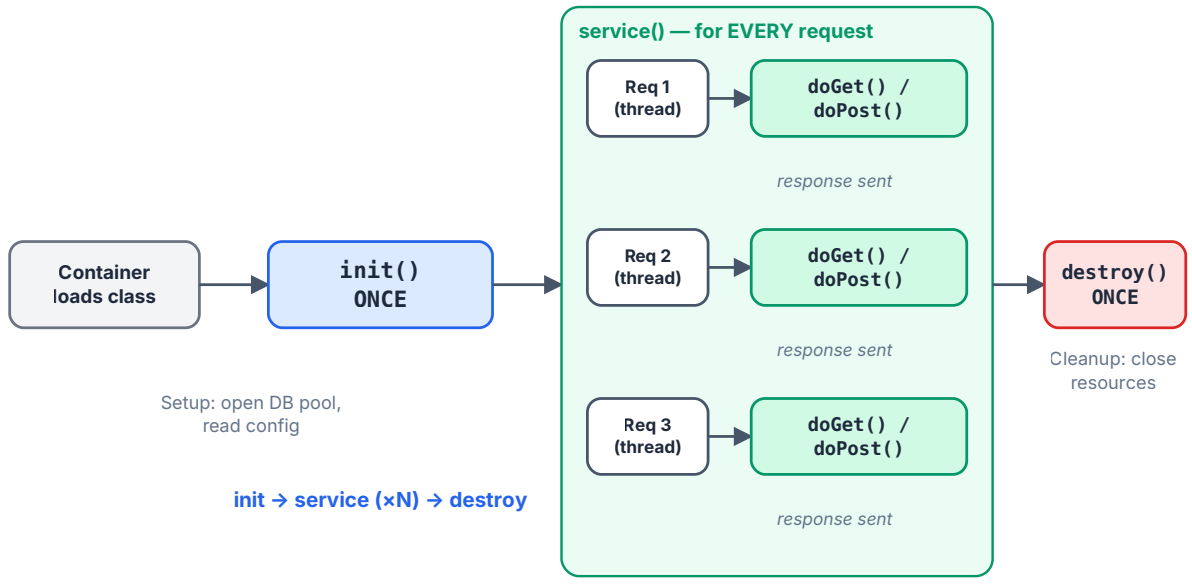


Fig. 22 — Life cycle of a servlet

```java
public class LifecycleServlet extends HttpServlet {
    public void init(ServletConfig config) throws ServletException { super.init(config); /* once */ }
    protected void doGet(HttpServletRequest q, HttpServletResponse r) throws IOException {
        r.getWriter().println("service()/doGet(): handling request");
    }
    public void destroy() { /* once - release resources */ }
}
```

💡 **Why it matters — Why this design is efficient**

`init()` runs only once, so expensive set-up (a database connection pool, loading configuration) is done a single time and reused for thousands of requests handled by `service()`. In CGI every request started a brand-new process.

🏭 **Real-life use case**

An **online exam portal**: one `LoginServlet` instance serves every student who logs in — 500 students at the same moment means 500 threads on the same object, not 500 processes. `init()` loads the question-paper configuration once; `destroy()` closes the pool when the server stops.

⚠️ **Careful**

Because many threads share one servlet object, do not keep per-user data in instance fields of a servlet — use local variables or the session.

📖 GFG: Servlet Life Cycle in Java

3.3 Servlet Packages, Classes, Interfaces and Methods

Package	Main members
`jakarta.servlet`	`Servlet`, `ServletConfig`, `ServletContext`, `ServletRequest`, `ServletResponse`, `GenericServlet` — protocol-agnostic
`jakarta.servlet.http`	`HttpServlet`, `HttpServletRequest`, `HttpServletResponse`, `HttpSession`, `Cookie`

Type	Key methods / role
`Servlet` (interface)	`init()`, `service()`, `destroy()`, `getServletConfig()`, `getServletInfo()`
`ServletRequest`	`getParameter()`, `getAttribute()`, `getInputStream()`
`ServletResponse`	`getWriter()`, `setContentType()`, `getOutputStream()`
`GenericServlet`	Abstract, protocol-independent base class implementing `Servlet`
`HttpServlet`	Abstract, HTTP-specific: `doGet`, `doPost`, …
`HttpSession`	Per-user, server-side session storage
`Cookie`	Small client-side key-value data

ServletConfig vs ServletContext

ServletConfig	ServletContext
Init parameters of **one** servlet (`<init-param>`)	Parameters / attributes of the **whole web application** (`<context-param>`)
Scope: that single servlet	Scope: shared by all servlets and JSPs in the app

📖 GFG: ServletConfig vs ServletContext

3.4 Handling Forms with a Servlet

```HTML
<form action="register" method="post">
  Name:  <input type="text"  name="username" />
  Email: <input type="email" name="email" />
  <input type="submit" value="Register" />
</form>
```

```JAVA
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        String username = req.getParameter("username");    // read by the 'name' attribute
        String email    = req.getParameter("email");
        resp.setContentType("text/html");
        resp.getWriter().println("<h2>Registered: " + username + " (" + email + ")</h2>");
    }
}
```

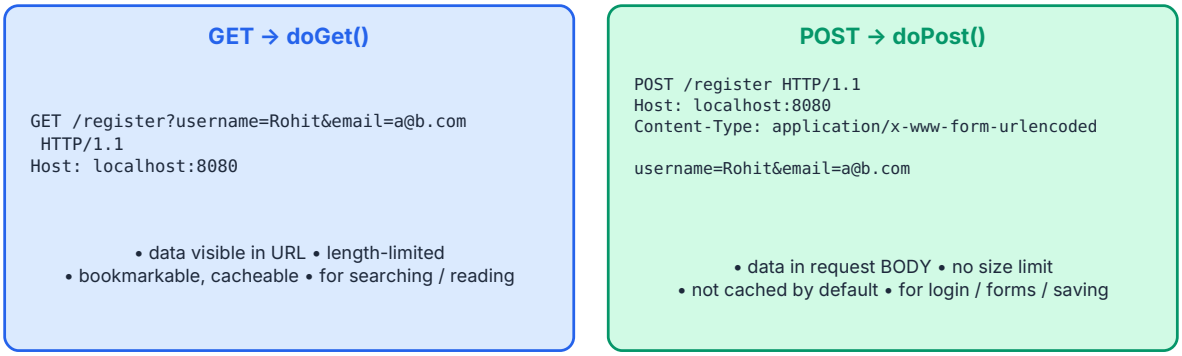


Fig. 23 — GET vs POST — where the form data travels

The `method` attribute of the `<form>` decides which of `doGet()` / `doPost()` runs.

🏭 Real-life use case

Search boxes use **GET** (the result page can be bookmarked and shared); login, registration and payment forms use **POST** so passwords and card data never appear in the URL or browser history.

📖 GFG: Handling Form Data in Servlet

3.5 Session Handling — Various Methods

Definition

Since HTTP is stateless, **session tracking** keeps a user's state / data across several requests. Servlets offer four techniques.

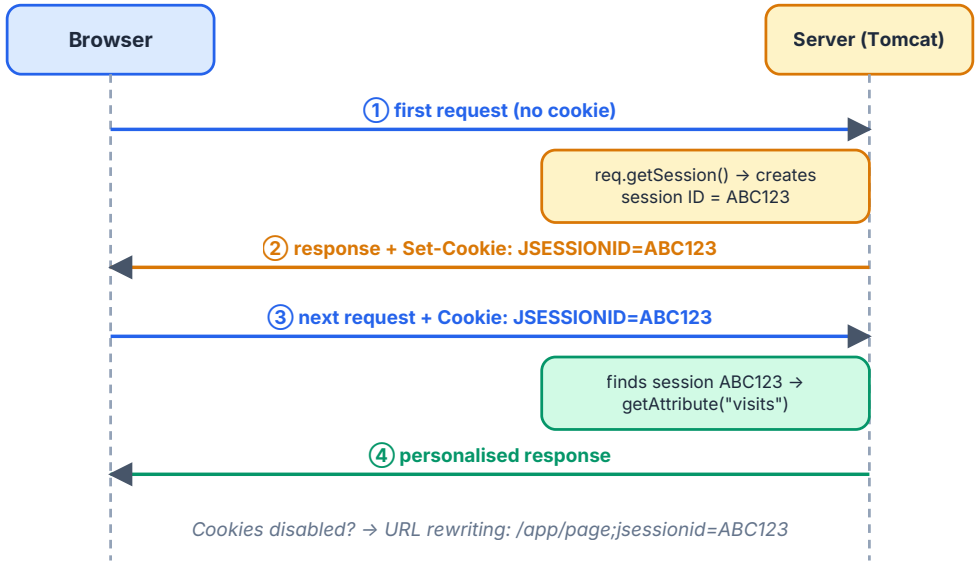


Fig. 24 — How HttpSession and the JSESSIONID cookie work together

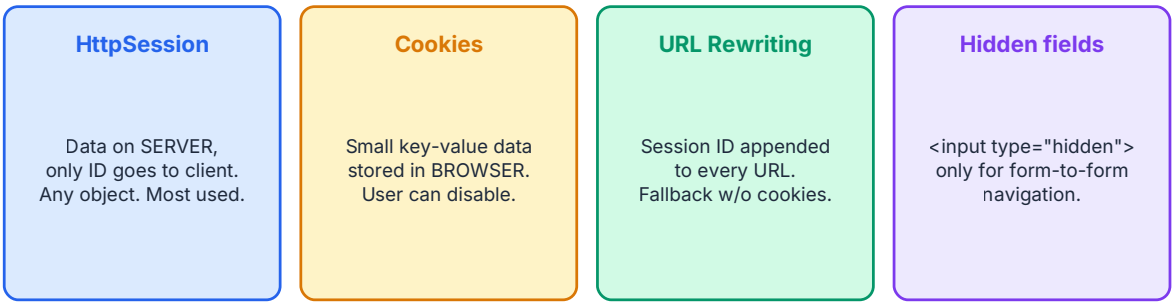


Fig. 25 — The four session-tracking techniques

```java
@WebServlet("/session-demo")
public class SessionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession();                // creates one if absent
        Integer visits = (Integer) session.getAttribute("visits");
        session.setAttribute("visits", visits == null ? 1 : visits + 1);

        Cookie c = new Cookie("lastVisit", String.valueOf(System.currentTimeMillis()));
        c.setMaxAge(60 * 60 * 24);   resp.addCookie(c);          // client-side cookie, 1 day

        String url = resp.encodeURL("/session-demo");            // URL rewriting fallback
        // hidden form field: <input type="hidden" name="uid" value="42">
    }
}
```

1. **HttpSession** — data kept on the server under a unique session ID (sent via the `JSESSIONID` cookie). Most powerful: stores any object.
2. **Cookies** — small key-value data stored in the browser and sent back with every request to that domain. Limited size; users can disable them.
3. **URL rewriting** — session ID appended to every URL; fallback when cookies are disabled.
4. **Hidden form fields** — invisible inputs resubmitted with each form; works only for form-driven navigation.

🏭 Real-life use case

An **e-commerce shopping cart**: items added by an anonymous visitor live in the `HttpSession` on the server; the browser holds only the `JSESSIONID`. A "remember me" login or "last visited" banner uses a long-lived `Cookie`. After login the app typically calls `session.invalidate()` on logout.

📖 GFG: Session Tracking in Servlet

3.6 Elements of Deployment Descriptors

📘 Definition

The **deployment descriptor** (`web.xml`) is an XML configuration file that tells the servlet container how to deploy and manage a web application — which servlets exist, which URLs map to them, session timeout, security rules — without changing Java code.

```xml
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="5.0">
  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>com.rohit.HelloServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
  <welcome-file-list><welcome-file>index.html</welcome-file></welcome-file-list>
  <session-config><session-timeout>30</session-timeout></session-config>   <!-- minutes -->
</web-app>
```

Element	Purpose
`<servlet>` / `<servlet-mapping>`	Register a servlet class / map a URL pattern to it
`<filter>` / `<filter-mapping>`	Pre / post-processing of requests (logging, auth)
`<listener>`	Life-cycle event listeners
`<context-param>`	Application-wide init parameters
`<welcome-file-list>`	Default page (index.html ...)
`<error-page>`	Custom pages per status code / exception
`<session-config>`	Session timeout settings
`<security-constraint>`	URL-based access control

📖 GFG: web.xml Deployment Descriptor

❓ Likely exam questions — Unit III

- HTTP is stateless — explain. Web server vs application server.
- Servlet life cycle with a diagram; why are servlets faster than CGI?
- GenericServlet vs HttpServlet; ServletConfig vs ServletContext.
- Write a servlet that reads a form and prints the values. GET vs POST.
- Explain all four session-tracking techniques.
- Explain the elements of web.xml with an example mapping.

UNIT IV · 8 HOURS

JSP & Database-Backed Web Applications

JSP life cycle | Directives | Scripting elements | Implicit objects | JDBC from JSP / Servlet | JavaBeans | Sessions | Error handling

4.1 JSP Basics

📘 **Definition**

JSP (JavaServer Pages) lets you embed Java code inside HTML. The container translates each JSP into a **Servlet** on the first request, so a JSP is really a servlet written in an HTML-friendly way.

JSP life cycle

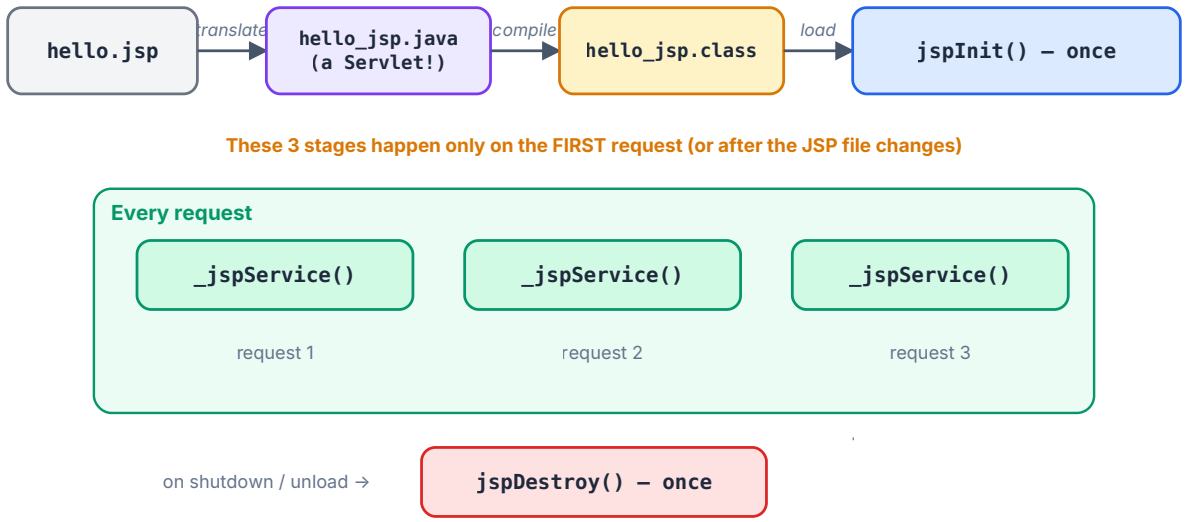


Fig. 26 — Translation and compilation happen once; _jspService() runs for every request

1. **Translation** — `.jsp` → servlet Java source (`_jsp.java`).
2. **Compilation** — Java source → `.class` .
3. `jspInit()` — called once (like a servlet's `init()`).
4. `_jspService()` — called for every request (auto-generated; handles GET / POST).
5. `jspDestroy()` — called once (like `destroy()`).

💡 **Why it matters — Why JSP when we have servlets?**

Writing HTML with `out.println()` inside a servlet becomes messy for big pages. JSP flips it: mostly HTML with small chunks of Java where data is dynamic. So **JSP = View** and **Servlet = Controller** (Unit V, MVC).

Directives

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.*, java.sql.*" %>
<%@ include file="header.jsp" %>
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

Scripting elements

Element	Syntax	Purpose
Scriptlet	`<% ... %>`	A block of Java statements
Expression	`<%= ... %>`	Evaluate and print the result into the HTML
Declaration	`<%! ... %>`	Declare a field or method at the class level of the generated servlet
Comment	`<%-- ... --%>`	Not sent to the client

```jsp
<%  int x = 10, y = 20;  %>
<p>Sum = <%= x + y %></p>
<%! int counter = 0;  int increment() { return ++counter; } %>
```

Standard actions

```jsp
<jsp:useBean id="student" class="com.rohit.Student" scope="session"/>
<jsp:setProperty name="student" property="name" value="Rohit"/>
<jsp:getProperty name="student" property="name"/>
<jsp:include page="footer.jsp"/>     <jsp:forward page="nextPage.jsp"/>
```

Implicit objects

📘 **Definition**

Implicit objects are created automatically by the container and available in every JSP without declaring them — they are local variables of the generated `_jspService()` method.

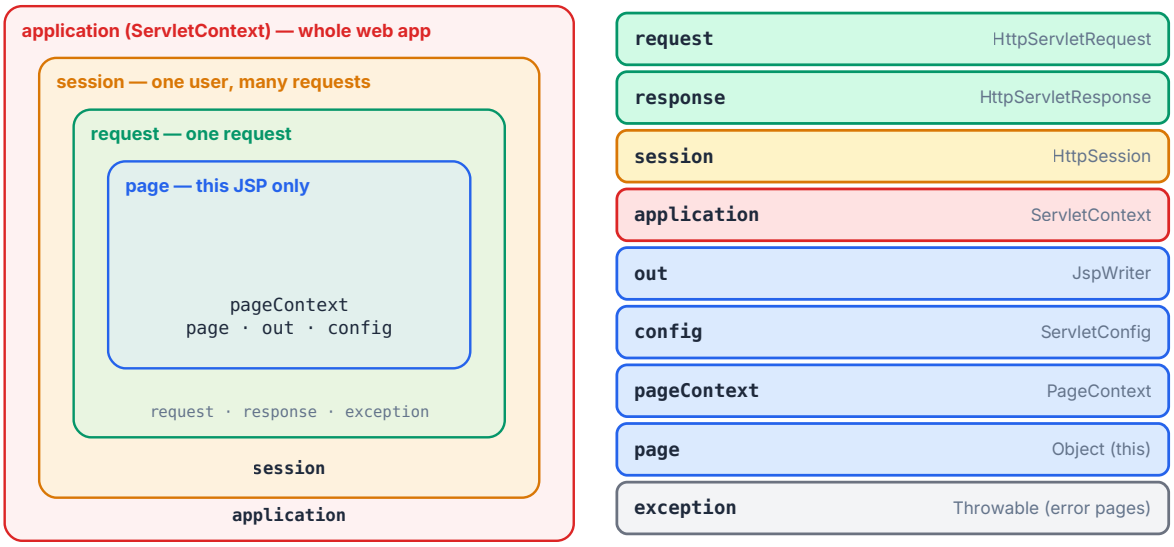


Fig. 27 — The nine implicit objects and the four scopes (page ⊂ request ⊂ session ⊂ application)

Object	Type	Purpose
`request`	HttpServletRequest	Incoming request data
`response`	HttpServletResponse	Outgoing response
`session`	HttpSession	Per-user session
`application`	ServletContext	App-wide shared data
`out`	JspWriter	Write output to the page
`config`	ServletConfig	Servlet init parameters
`pageContext`	PageContext	Access to all other scopes
`page`	Object (`this`)	The current JSP instance
`exception`	Throwable	Only in error pages (`isErrorPage="true"`)

```jsp
<html><body>
<%  String name = request.getParameter("name");
    session.setAttribute("user", name);  %>
<h2>Hello, <%= name %>! Session ID: <%= session.getId() %></h2>
</body></html>
```

⭐ Exam tip

There are **9** implicit objects — list all nine with their types; partial answers lose easy marks.

🧠 Yaad rakho

R-R-S-A-O-C-P-P-E → request, response, session, application, out, config, pageContext, page, exception.

📖 GFG: JSP Implicit Objects

4.2 JSP / Servlet Database Connectivity — Oracle, MS-SQL, MySQL

💡 Why it matters — The key idea

The JDBC API (`java.sql.*`) is **identical** for every vendor. Only the **connection URL** and the **driver JAR** (placed in `WEB-INF/lib`) change — that vendor independence is JDBC's design goal.

```java
static Connection mysql()  throws SQLException {
    return DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "root", "password"); }
static Connection oracle() throws SQLException {
    return DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:XE", "system", "password"); }
static Connection mssql()  throws SQLException {
    return DriverManager.getConnection("jdbc:sqlserver://localhost:1433;databaseName=mydb", "sa",
"password"); }
```

Database	URL pattern	Driver JAR
MySQL	`jdbc:mysql://host:3306/db`	mysql-connector-j
Oracle	`jdbc:oracle:thin:@host:1521:XE`	ojdbc11.jar
MS-SQL Server	`jdbc:sqlserver://host:1433;databaseName=db`	mssql-jdbc

Querying, inserting, deleting and modifying records

```java
try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    // SELECT
    try (PreparedStatement ps = conn.prepareStatement("SELECT * FROM students WHERE cgpa > ?")) {
        ps.setFloat(1, 8.0f);   ResultSet rs = ps.executeQuery();
        while (rs.next()) System.out.println(rs.getString("name"));
    }
    // INSERT
    try (PreparedStatement ps = conn.prepareStatement("INSERT INTO students (name, cgpa) VALUES (?, ?)"))
{
        ps.setString(1, "Anjali");  ps.setFloat(2, 9.1f);  ps.executeUpdate();
    }
    // UPDATE
    try (PreparedStatement ps = conn.prepareStatement("UPDATE students SET cgpa = ? WHERE name = ?")) {
        ps.setFloat(1, 9.5f);  ps.setString(2, "Anjali");  ps.executeUpdate();
    }
    // DELETE
    try (PreparedStatement ps = conn.prepareStatement("DELETE FROM students WHERE name = ?")) {
        ps.setString(1, "Anjali");  ps.executeUpdate();
    }
}
```

Types of Statement

Type	Use
`Statement`	Static SQL without parameters — vulnerable to SQL injection; avoid for user input
`PreparedStatement`	Pre-compiled, parameterised (`?` placeholders) — faster on repeat, injection-safe
`CallableStatement`	Calls stored procedures: `{call procName(?, ?)}`

💡 Why it matters — How PreparedStatement stops SQL injection

With `Statement`, user input is glued into the SQL text, so input like `' OR '1'='1` changes the logic of the query. With `PreparedStatement`, the SQL structure is compiled first with `?` placeholders and the values are bound afterwards as **pure data** — they can never be read as SQL syntax.

🏭 Real-life use case

A **login page**: `SELECT * FROM users WHERE email = ? AND password_hash = ?` with a PreparedStatement. A hacker typing `admin' --` in the email box just fails to find a user, instead of bypassing the password check.

📖 GFG: JDBC PreparedStatement vs Statement

4.3 Separating Business Logic and Presentation Logic

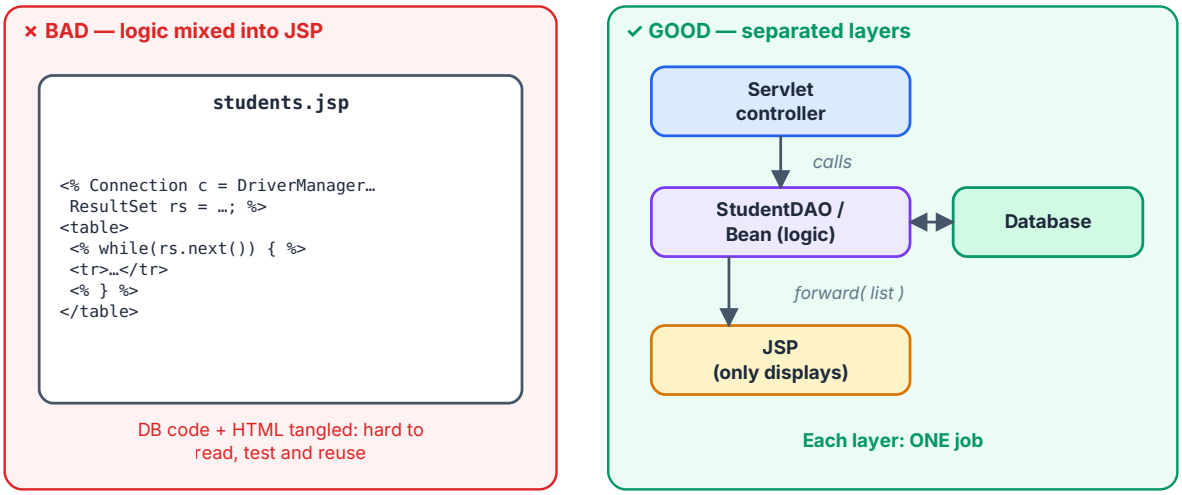


Fig. 28 — Database code mixed into a JSP (bad) vs layered design (good)

💡 Why it matters — Why separate?

Mixing DB / business code into a JSP makes the page hard to read, hard to maintain and impossible to reuse — it violates **separation of concerns**. Move logic into JavaBeans / DAO classes, orchestrate them from a servlet, and let the JSP only **render**. This is the foundation of MVC (Section 5.1).

Building and using a JavaBean

📘 Definition

A **JavaBean** is a plain reusable Java class with a **public no-argument constructor**, **private fields**, public **getters / setters** for each property, and ideally implements `Serializable`.

```java
package com.rohit.beans;
import java.io.Serializable;
public class StudentBean implements Serializable {
    private String name;  private float cgpa;
    public StudentBean() { }                                  // required no-arg constructor
    public String getName()          { return name; }
    public void   setName(String n)  { this.name = n; }
    public float  getCgpa()          { return cgpa; }
    public void   setCgpa(float c)   { this.cgpa = c; }
}
```

```jsp
<jsp:useBean id="student" class="com.rohit.beans.StudentBean" scope="page"/>
<jsp:setProperty name="student" property="*"/>          <%-- auto-binds matching request params --%>
<p>Name: <jsp:getProperty name="student" property="name"/></p>
<p>CGPA: <jsp:getProperty name="student" property="cgpa"/></p>
```

⭐ **Exam tip — Naming convention (list for full marks)**

For a private field `name`: getter `getName()` (or `isName()` for boolean) and setter `setName(String)`. This strict naming lets `<jsp:setProperty property="*"/>` match request parameters to bean properties by reflection.

🏭 **Real-life use case**

A **hospital app**: `PatientBean` holds the data, `PatientDAO` contains all SQL, and `patientList.jsp` only loops over a list and prints a table. When the database changes from MySQL to Oracle, only the DAO changes — no JSP is touched.

📖 GFG: JavaBeans in Java

4.4 Session Handling in JSP

Same mechanisms as servlets (Section 3.5), accessed through the implicit `session` object.

```jsp
<%  session.setAttribute("username", request.getParameter("username"));
    session.setMaxInactiveInterval(30 * 60);      // 30 minutes, in seconds   %>
Welcome back, <%= session.getAttribute("username") %>!
```

🏭 **Real-life use case**

After login a user sees "Welcome, Rohit" on every page without logging in again; each page begins with a check such as `if (session.getAttribute("username") == null) response.sendRedirect("login.jsp");` to protect private pages. `session.invalidate()` on logout clears everything.

📖 GFG: Session Handling in JSP

4.5 Types of Errors and Exception Handling (JSP)



Fig. 29 — Three kinds of JSP errors and how each is handled

```jsp
<%-- errorPage.jsp --%>
<%@ page isErrorPage="true" %>
<h2>An error occurred: <%= exception.getMessage() %></h2>

<%-- source.jsp - declares which page handles ITS errors --%>
<%@ page errorPage="errorPage.jsp" %>
<%  int x = 10 / 0;  %>          <%-- ArithmeticException -> forwarded to errorPage.jsp --%>
```

1. **Translation-time errors** — bad JSP syntax (mismatched tags, invalid directives); the container cannot translate the page into a servlet.
2. **Compilation-time errors** — invalid Java inside scriptlets / expressions; generated servlet fails to compile.
3. **Runtime exceptions** — `NullPointerException`, `SQLException`, `ArithmeticException` ...; handled with `errorPage` / `isErrorPage` for one page, or `<error-page>` in `web.xml` for application-wide status-code pages (custom 404 / 500).

🏭 Real-life use case

Instead of showing users a scary stack trace, an e-commerce site forwards every runtime error to one friendly `error.jsp` ("Something went wrong — order #... was not charged") while the exception is written to the server log for developers.

📖 GFG: Exception Handling in JSP

❓ Likely exam questions — Unit IV

- Explain the JSP life cycle with a diagram. Why is a JSP called a servlet?
- Scriptlet vs expression vs declaration. List all nine implicit objects.
- Write JSP / servlet code to insert and display records using JDBC (any one database).
- Statement vs PreparedStatement vs CallableStatement; how does PreparedStatement prevent SQL injection?
- What is a JavaBean? Explain useBean / setProperty / getProperty.
- Types of errors in JSP and how to handle them.

UNIT V · 8 HOURS

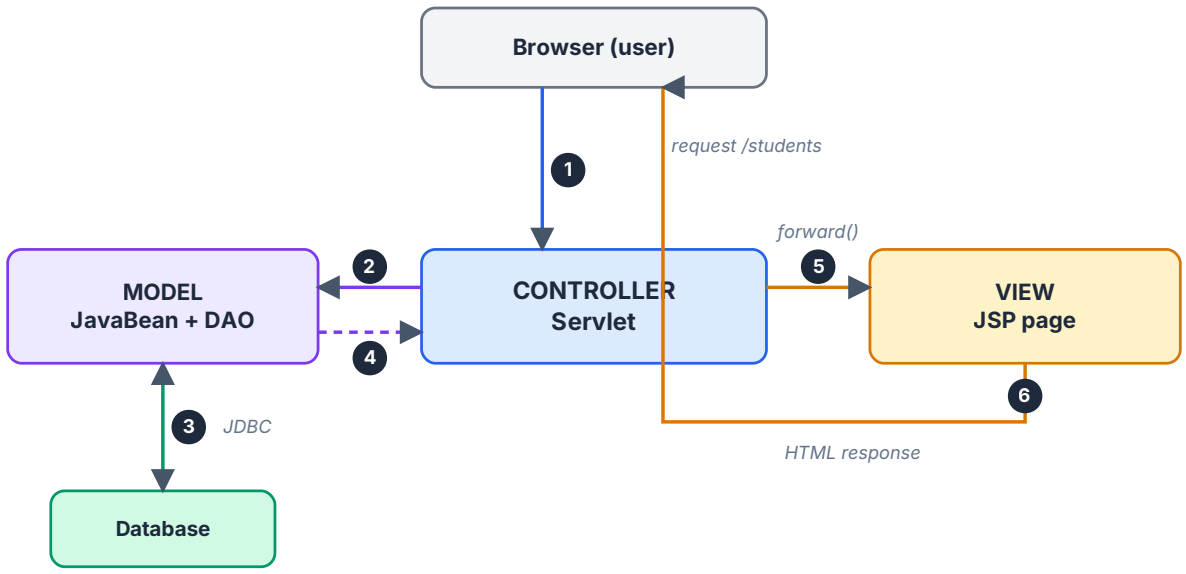
MVC, RMI & EJB

MVC architecture | Controller / Model / View | RMI | Stub & skeleton | EJB | Session beans | Message-driven beans | Container services

5.1 MVC Architecture

Definition

MVC (Model-View-Controller) is a design pattern that splits an application into three interconnected parts so each can be developed, tested and changed independently.



①request ②call model ③query DB ④return data (setAttribute) ⑤forward to JSP ⑥HTML to browser

Model never knows about HTML · View never talks to DB · Controller just coordinates

Fig. 30 — Request flow in a servlet / JSP MVC application

Component	Implemented as	Responsibility
Model	JavaBeans + DAO classes	Data and business logic. Knows nothing about how data is displayed.
View	JSP pages	Presentation only — renders HTML from the data it is given; no business logic.
Controller	Servlet	Receives the request, calls the Model, then forwards to the right View with `RequestDispatcher.forward()`.

```java
@WebServlet("/students")
public class StudentController extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        StudentDAO dao = new StudentDAO();                      // Model
        List<StudentBean> students = dao.getAllStudents();
        req.setAttribute("students", students);                 // hand data to the view
        req.getRequestDispatcher("studentList.jsp").forward(req, resp);   // View
    }
}
```

Benefits of MVC

1. **Separation of concerns** — each layer has one clear job.
2. **Easier testing** — business logic can be unit-tested without a UI.
3. **Parallel development** — front-end and back-end teams work at the same time.
4. **Reusability** — one Model can serve several Views (web page, mobile app).

🏭 Real-life use case

A **hospital management system**: the receptionist's browser requests `/patients`; `PatientController` (servlet) asks `PatientDAO` (model) for records and forwards them to `patients.jsp` (view). A mobile app later reuses the same model with a JSON view — no business code is rewritten. Frameworks like Spring MVC and Struts are exactly this pattern.

🧠 Yaad rakho

Controller traffic police hai — gaadi (request) ko kahan bhejna hai wo decide karta hai; **Model** kaam karta hai, **View** sirf dikhata hai.

📖 GFG: MVC Architecture in Java

5.2 Introduction to Remote Method Invocation (RMI)

📘 Definition

RMI is a Java API that lets an object in one JVM call a method on an object in a different JVM — possibly on another machine — so the remote call looks like a normal local call.

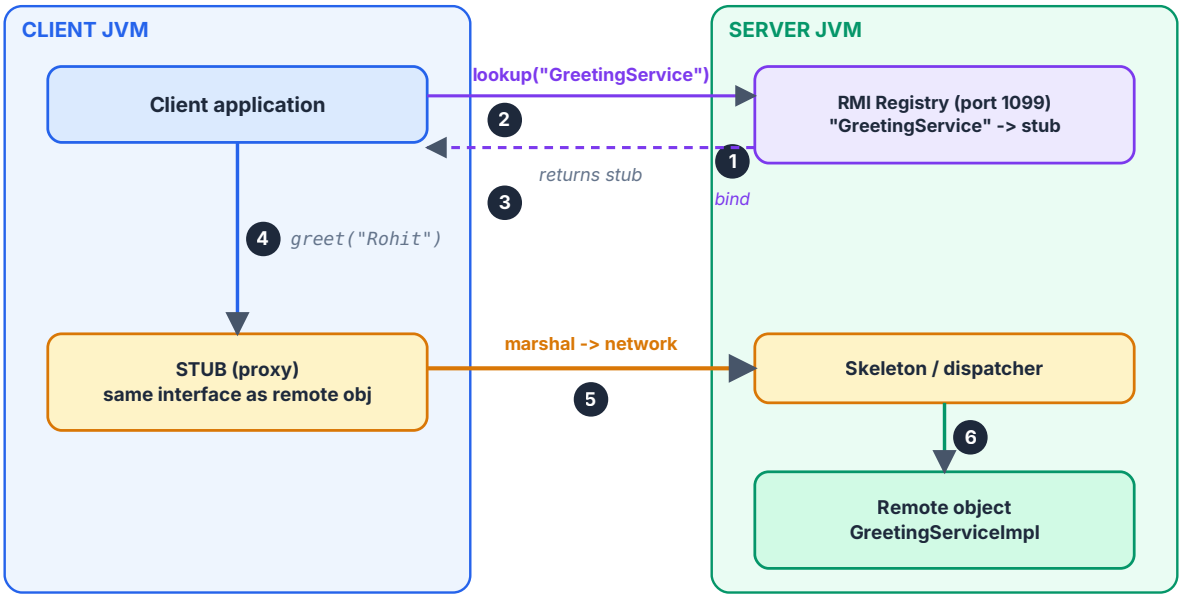


Fig. 31 — RMI architecture — registry, stub, network and the real remote object

- **Stub** — client-side proxy with the same interface as the remote object; it *marshals* (packs) the call and sends it over the network.
- **Skeleton / dispatcher** — server side; *unmarshals* the call and invokes the real method (modern Java generates this automatically).
- **RMI Registry** — a phone-book style lookup service where server objects are bound under a name and clients look them up.

Steps to build an RMI application

1. Define a **remote interface** extending `java.rmi.Remote`; every method `throws RemoteException`.
2. Implement it in a server class, usually extending `UnicastRemoteObject`.
3. Register the object with the RMI registry — `Registry.rebind()`.
4. On the client, look up the object by name and call its methods as if local.

```java
// 1. Remote interface
public interface GreetingService extends Remote {
    String greet(String name) throws RemoteException;
}
// 2. Implementation
public class GreetingServiceImpl extends UnicastRemoteObject implements GreetingService {
    protected GreetingServiceImpl() throws RemoteException { super(); }
    public String greet(String name) throws RemoteException { return "Hello, " + name + "! (from the
remote server)"; }
}
// 3. Server
Registry registry = LocateRegistry.createRegistry(1099);
registry.rebind("GreetingService", new GreetingServiceImpl());
// 4. Client
Registry reg = LocateRegistry.getRegistry("localhost", 1099);
GreetingService svc = (GreetingService) reg.lookup("GreetingService");
System.out.println(svc.greet("Rohit"));
```

🏭 Real-life use case

A **bank's branch software** calls `account.withdraw(5000)` on an account object that actually lives on the head-office server — the branch code does not open sockets or parse messages; RMI hides the network. Other uses: distributed task scheduling, remote monitoring tools. (Today REST and gRPC are more common, but RMI is the classic Java-to-Java way.)

⭐ Exam tip

Draw: Client → Stub → Network → Skeleton → Remote object, with the Registry on the side, and number the five steps (bind, lookup, get stub, call, return).

📖 GFG: RMI in Java

5.3 Introduction to Enterprise JavaBeans (EJB)

📘 Definition

EJB is a server-side, managed component model (part of Jakarta EE) for distributed, transactional and secure business logic. The **EJB container** handles concurrency, transactions, security and object life cycle, so developers write only business logic.

Types of EJB

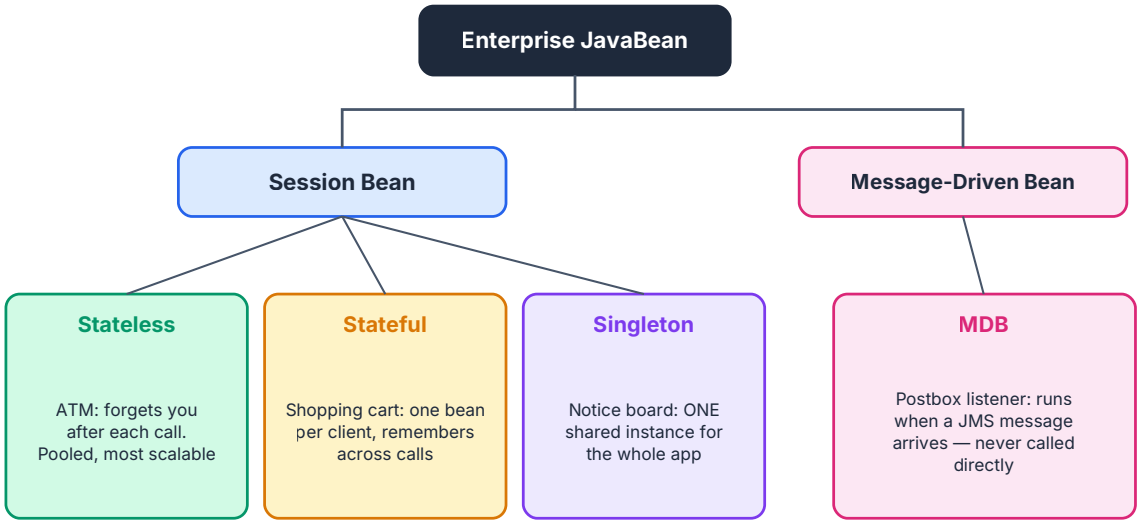


Fig. 32 — Session beans (stateless, stateful, singleton) and message-driven beans

Type	Behaviour	Analogy / example
Stateless session bean	No client state kept between calls; instances pooled and reused — most scalable.	ATM machine — `CalculatorService.add(a, b)`
Stateful session bean	One instance per client; remembers conversational state across calls.	Shopping cart
Singleton session bean	Exactly ONE instance shared by the whole application.	Application-wide cache or counter
Message-driven bean (MDB)	Never called directly; triggered asynchronously by a JMS message.	Order-processing queue

Entity beans (EJB 2.x) are deprecated in favour of JPA.

```java
@Local public interface CalculatorService { int add(int a, int b); }

@Stateless
public class CalculatorServiceBean implements CalculatorService {
    public int add(int a, int b) { return a + b; }
}

@Stateful
public class ShoppingCartBean {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }       // remembered across calls
    public List<String> getItems()   { return items; }
}

@WebServlet("/calc")
public class CalcServlet extends HttpServlet {
    @EJB private CalculatorService calculator;                   // container injects it
    protected void doGet(HttpServletRequest q, HttpServletResponse r) throws IOException {
        r.getWriter().println("Result: " + calculator.add(5, 10));
    }
}
```

Services provided by the EJB container

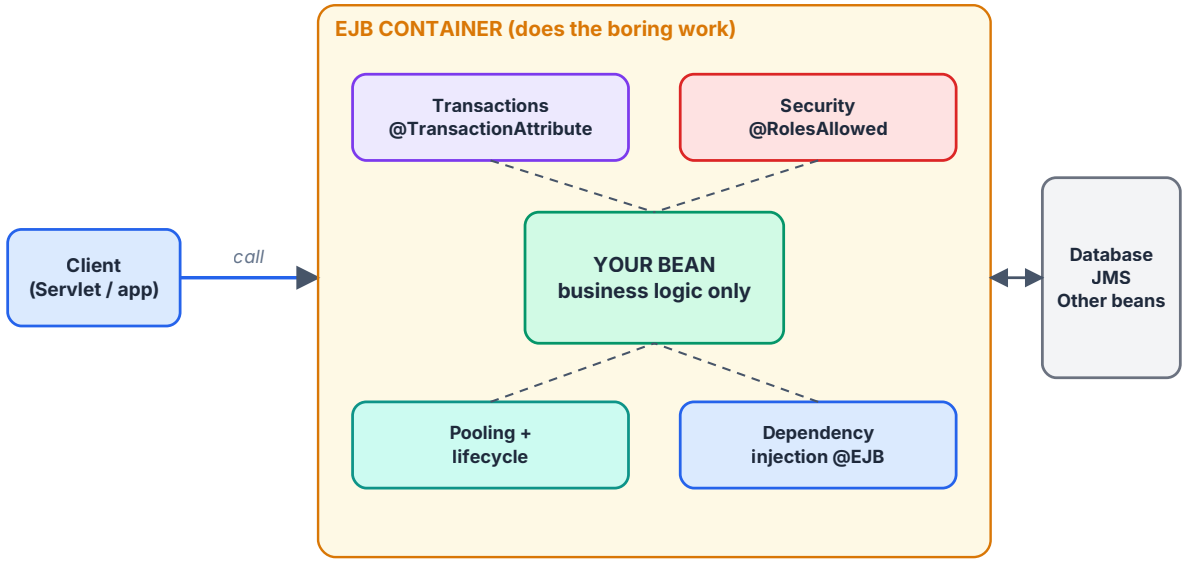


Fig. 33 — The container wraps your bean with transactions, security, pooling and injection

1. **Declarative transactions** — `@TransactionAttribute`; no hand-written commit / rollback.
2. **Declarative security** — `@RolesAllowed`.
3. **Instance pooling and life-cycle management**.
4. **Remote / local invocation transparency**.
5. **Dependency injection** — `@EJB`, `@Resource`.

RMI vs EJB

RMI	EJB
Low-level remote invocation mechanism	High-level managed component model built on remote-invocation ideas
No built-in transactions / security — code them yourself	Container supplies transactions, security, pooling
Developer manages object life cycle	Container manages bean life cycle

🏭 Real-life use case

Online banking: `TransferService` is a **stateless** bean — `@TransactionAttribute(REQUIRED)` makes "debit A + credit B" all-or-nothing automatically. The checkout cart is a **stateful** bean. A global exchange-rate cache is a **singleton**. After payment, an **MDB** listens on a JMS queue and sends the SMS / e-mail confirmation asynchronously so the customer never waits.

📖 GFG: Enterprise Java Beans (EJB)

❓ Likely exam questions — Unit V

- Explain MVC with a neat diagram; role of Model, View and Controller; advantages.
- Explain the RMI architecture (stub, skeleton, registry) and the steps to build an RMI application with code.
- What is EJB? Types of EJB with examples; services provided by the container.
- Stateless vs stateful session beans; what is a message-driven bean?
- Compare RMI and EJB.

Quick comparison sheet

One-glance tables for the last day before the exam. Each row is a favourite "differentiate between" question.

Core Java and OOP

Topic	Remember
JDK / JRE / JVM	JDK ⊃ JRE ⊃ JVM — develop+run / run / execute bytecode
String / Buffer / Builder	Immutable / mutable + thread-safe / mutable + fastest
== vs equals()	Reference vs content
Widening / narrowing	Automatic & safe / explicit cast & lossy
Overloading / overriding	Same class, different params, compile time / subclass redefines, runtime
Abstract class / interface	One parent, has constructor / many allowed, no constructor
Checked / unchecked	Compiler forces handling / runtime bugs
throw / throws	Raise an exception / declare in signature
start() / run()	New thread / ordinary method call
Thread / Runnable	Extend (uses up inheritance) / implement (preferred)

Database, web and enterprise

Topic	Remember
JDBC steps	Driver → Connection → Statement → Execute → ResultSet → Close
Statement / Prepared / Callable	Static SQL / parameterised, safe / stored procedures
Web server / app server	Static content / dynamic Java, business logic
GET / POST	URL, limited, cacheable / body, no limit, hidden
GenericServlet / HttpServlet	Any protocol, override service() / HTTP, doGet / doPost
ServletConfig / ServletContext	One servlet / whole application
Servlet life cycle	init once → service per request → destroy once
JSP life cycle	Translate → compile → jspInit → _jspService → jspDestroy
Session tracking	HttpSession, Cookies, URL rewriting, Hidden fields
Scriptlet / expression / declaration	`<% %>` statements / `<%= %>` output / `<%! %>` class-level members
MVC	Model = data + logic (Bean/DAO), View = JSP, Controller = Servlet
RMI	Remote interface → implementation → rebind in registry → client lookup
EJB types	Stateless (pooled), Stateful (per client), Singleton (one), MDB (JMS-triggered)
RMI / EJB	Low-level remote call / managed components with transactions & security

Exam-day revision checklist

Tick these off — if you can **write or draw** each one from memory, you cover roughly 70% of a typical M.Tech Java paper.

- ☐ Features of Java (top 6, two lines each)
- ☐ JDK vs JRE vs JVM — Fig. 4
- ☐ Primitive data types table (size, default, range)
- ☐ JVM memory areas — heap, stack, method area, PC, native
- ☐ Widening vs narrowing conversion
- ☐ String vs StringBuffer vs StringBuilder table
- ☐ Why String is immutable (4 reasons)
- ☐ Method overloading resolution order
- ☐ Access-modifier table
- ☐ Checked vs unchecked exceptions + hierarchy figure
- ☐ try-catch-finally flow
- ☐ Thread creation: extends Thread vs implements Runnable
- ☐ Thread life cycle diagram + state table
- ☐ Synchronization — race condition and lock
- ☐ JDBC driver types (1–4) and steps to connect
- ☐ Web server vs application server
- ☐ Servlet life cycle: init → service → destroy
- ☐ Session tracking: all four methods
- ☐ JSP life cycle and all 9 implicit objects
- ☐ Statement vs PreparedStatement vs CallableStatement
- ☐ JavaBean rules and useBean / setProperty / getProperty
- ☐ MVC diagram + role of each layer
- ☐ RMI architecture + 4 steps to build an RMI app
- ☐ EJB types (Stateless / Stateful / Singleton / MDB) with analogies

Recommended resources

Resource	Covers
Java: The Complete Reference — Herbert Schildt	Units I & II
Head First Servlets & JSP	Units III & IV
Head First EJB · Oracle / Jakarta EE Tutorial	Unit V
docs.oracle.com/javase — official Oracle docs	Core Java API reference
jakarta.ee/specifications — official specs	Servlet, JSP, EJB specifications
tomcat.apache.org — official Tomcat docs	Server set-up and deployment
geeksforgeeks.org/java — Java tutorial hub	Everything, unit-by-unit, with practice questions

All diagrams in this book were drawn for this book; the per-topic "GFG" links open GeeksforGeeks search results for extra reading and practice problems.